arXiv:2602.02898v5 [cs.AI] 1 Oct 2026

Aligning Language Model Benchmarks with Pairwise Preferences

Marco Gutierrez[1] sgw3fy@virginia.edu **Xinyi Leng**[1] **Hannah Cyberey**[1]

Jonathan Richard Schwarz[2] **Ahmed Alaa**[3]

Thomas Hartvigsen[1]

[1] School of Data Science, University of Virginia
[2] Imperial College London
[3] Department of Electrical Engineering and Computer Science, UC Berkeley and UCSF

Abstract

Language model benchmarks are pervasive and computationally-efficient proxies for real-world downstream performance. However, many recent works find that benchmarks often fail to predict downstream utility. While some works have begun diagnosing sources of misalignment, there remain no ways to systematically update benchmarks to align their scores with downstream usage. Towards bridging this gap, we introduce and study *benchmark alignment*, where we use information about downstream model performance to automatically update benchmarks, specifically aiming to update static benchmarks so they generalizably rank models according to new pairwise preferences. Our experiments involving 4576 language models and 6 benchmarks show that reweighting benchmark items can successfully rank unseen models, even generalizing across model scales in most cases. And while naive alignment unsurprisingly requires large numbers of models and benchmark questions, an oracle experiment suggests this could be reduced to as few as 20 well-chosen models. Overall, our work takes a step towards efficiently aligning benchmark development with downstream tasks.[1]

1 Introduction

Benchmarks are the standard way to track progress in language modeling, operating on the assumption that aggregated test scores directly predict downstream performance. For example, the top-scoring model on a *language understanding benchmark* is presumed to be the *best language understanding model*. However, many recent works have found that the best models on a benchmark often underperform in deployment, even for the same task (Alaa et al., 2025; Salaudeen and Hardt, 2025; Zhang et al., 2025a). Similarly, general-purpose models, despite excelling on general-capability benchmarks, often perform worse than smaller task-specific models (Pecher et al., 2025; Zhou et al., 2025), suggesting that benchmark scores are poor predictors of real-world utility.

Recent works have begun studying ways to improve benchmarks, many of which refine benchmarks to better capture model capabilities. These methods select informative subsets of test items (Polo et al., 2024; Vivek et al., 2024; Kipnis et al., 2025), often drawing from psychometrics using item response theory (IRT) (Vania et al., 2021; Hofmann et al., 2025; Kipnis et al., 2025). The resulting subsets can

[1] All of our code, models, and data are publicly-available: `https://github.com/hartvigsen-group/benchalign`.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Track on Evaluations and Datasets.

then be inspected for patterns that explain what the benchmark measures. While understanding the signals captured by benchmarks is important, these approaches leave two important, open questions: 1) *Do their rankings reflect real-world utility?* and 2) *Can they be updated to better predict utility for specific downstream uses?*

Downstream utility is extremely challenging to estimate. One common approach is to use pairwise preferences from downstream users: Given two models, practitioners choose which they prefer for their specific task (Wang et al., 2023; Cui et al., 2024). For benchmarking, the most important signal is how models rank against one another rather than the raw performance. For example, in high-stakes settings like healthcare, models that score well on standard benchmarks can still produce outputs that are inaccurate or unsafe when evaluated against clinician preferences (Arora et al., 2025; Busch et al., 2025). Yet there remains no systematic way to assess whether and how existing benchmarks can be aligned to reflect these preferences.

We introduce *benchmark alignment*: given a benchmark dataset and a set of language models with an externally specified preference ranking (e.g., derived from human evaluations), we ask whether existing benchmarks can be reweighted to induce rankings that better reflect these preferences while generalizing to new models. This task is inherently challenging, as most existing benchmarks are not designed to encode external preferences and are static, even if they fail to predict utility.. To study whether benchmark alignment is feasible, we propose a simple learning-to-rank approach as a diagnostic tool. Our approach learns preference-aligned weights for individual test items by predicting model pairwise rankings. These weights are interpretable and quantify the relevance of each test item to the target preferences. The reweighted benchmark items then produce an aggregate score that can be used to rank unseen models. Because this method is straightforward, it lets us examine relationships between alignment, preference data volume, and model selection.

In experiments spanning 4576 models, 6 benchmarks and 7 reward models, we find that benchmark alignment is surprisingly feasible. Aligned benchmarks accurately rank models unseen during training, including substantially larger models (30-70B+ parameters), across preferences of helpfulness and honesty. However, large volumes of models and questions are required to achieve alignment naively, highlighting data requirements as a key challenge for benchmark alignment. Interestingly, we find that an oracle can use as few as 20 models to achieve high-quality alignment, suggesting that identifying them ahead of time could be an important future step.

In summary, our contributions are as follows:

- We propose aligning benchmarks with pairwise preferences, alongside a straightforward method to study benchmark alignability (Section 3.2).
- We show that benchmark alignment is feasible using only small and medium-sized model data, with re-weighted benchmarks generalizing rankings to substantially larger models (Section 4.2)
- We find that naive alignment is data-intensive, requiring around 1,000 models and 5,000 questions, though as few as 20 carefully chosen models can suffice (Section 4.3).
- We show that benchmark distillation methods can reduce the data requirements for alignment to using only 1,500 questions ($\sim$7% of OpenLLMLeaderboard), with minimal loss in ranking accuracy. (Section 4.4)

2 Related Work

Benchmark Validity and Utility. Static benchmarks have long been the standard for assessing model capabilities and tracking progress in the field. However, numerous studies have raised concerns about their *reliability*–whether they yield consistent results–and *validity*–whether they measure what they purport to measure (Bowman and Dahl, 2021; Subramonian et al., 2023; Reuel et al., 2024; Saxon et al., 2024). In particular, current benchmarks often lack *external validity* (or predictive validity) where improvements in benchmark performance do not reliably translate into improved real-world utility (Liao et al., 2021). For instance, Alaa et al. (2025) empirically show that performance on MedQA, a benchmark intended to evaluate medical knowledge, correlates weakly with models' actual clinical performance. This evaluation gap undermines the use of benchmarks for model selection, as practitioners cannot confidently assume that higher benchmark rankings correspond to better use experiences or downstream performance.

Refining Benchmarks. To improve the predictive validity of benchmarks, recent work has proposed methods to refine existing benchmarks. Several studies apply Item Response Theory (IRT), a psychometric technique that models item difficulty and discrimination, to identify smaller subsets of informative benchmark questions that more efficiently and accurately measure model capabilities (Polo et al., 2024; Kipnis et al., 2025). Hofmann et al. (2025) introduce fluid benchmarking, which combines IRT with adaptive testing to dynamically select evaluation items matched to a model's capability. While these approaches focus on improving the measurement of latent model capabilities, our work instead aligns benchmarks with downstream user preferences that may not be fully captured by capability-driven metrics.

Alignment Evaluation and System Ranking. While human annotations remain the gold standard for evaluation, constructing a large-scale benchmark or leaderboard like Chatbot Arena that relies on actual human ratings is costly. The use of LLM-based judges, which uses frontier models to approximate human judgment, has become widely adopted, especially for evaluating model alignment in open-ended tasks (Zheng et al., 2023; Dubois et al., 2023; Li et al., 2024; Kim et al., 2024). To compare overall model quality, a system-level ranking is typically derived by aggregating over multiple instance-level evaluations produced by these judges (Gera et al., 2025). However, prior works on LLM-based evaluators have only assessed their performance at the instance level. Gao et al. (2025) find that LLM judges that perform well on per-example evaluations can produce substantially different and sometimes misaligned system-level rankings. Moreover, LLM judges are prone to systematic biases, such as positional bias (Wang et al., 2024c) and self-preference bias (Liu et al., 2024). Our work also targets system-level ranking, but instead of relying on LLM judges, we apply learning-to-rank methods to learn preference-aligned weightings for benchmark questions, optimizing predictive validity against human-preferred model rankings.

3 Benchmark Alignment

3.1 Problem Definition

Let $\mathcal{F} = \{f_1, f_2, ..., f_K\}$ be a set of language models and $\mathcal{D} = (Q, s)$ be a benchmark, which contains a set of textual test items $Q = \{q_1, ..., q_M\}$ and a scoring function s. Drawing from educational testing (Hively et al., 1968) and psychometrics (Embretson, 1998), we can treat each test item as a *stimulus* designed to elicit model responses that carry information about the underlying construct (e.g., model abilities) we wish to measure for a given task.

The scoring function produces a scalar score $s(f_i, Q) \in \mathbb{R}$ for model f_i that measures its overall performance on the benchmark, where higher is better (e.g., accuracy). Based on the scores $s(f_i, Q)$ produced for each model $f_i \in \mathcal{F}$, we can rank the models on benchmark $\mathcal{D}$ by sorting them based on their scores: $R_{\mathcal{D}} : \mathcal{F} \to \{1, 2, ..., K\}$, where $R_{\mathcal{D}}(f_i)$ is f_i's rank on benchmark $\mathcal{D}$ based on its score $s(f_i, Q)$. For example, if $R_{\mathcal{D}}(f_i) = 4$, then model f_i is the 4th best model.

Now suppose there exists a target pairwise preference over the models in $\mathcal{F}$ (e.g., perceived helpfulness, honesty), denoted by $f_i \succ_T f_j$, such that it induces a target pairwise ranking $R_T(f_i)$. Our goal is to construct a new benchmark $\hat{\mathcal{D}} = (\hat{Q}, s)$ that accurately ranks models according to the target preferences, such that the induced ranking $R_{\hat{\mathcal{D}}}(f_i) = R_T(f_i)$. A benchmark $\hat{\mathcal{D}}$ that ranks models according to R_T is thus *aligned* with the target rankings. The central question we study is: *to what extent can existing benchmarks be aligned with target preferences, and under what conditions does this alignment hold?*

3.2 Studying Alignment via Learning to Rank

While many approaches could be used to study alignment, the objective is not to propose a definitive solution to benchmark alignment. Instead, our goal is to examine whether preference-aligned weightings for benchmark questions can be learned from limited data, and whether the resulting rankings generalize to unseen models. We refer to this approach as BENCHALIGN[2].

Benchmark Construction. Standard benchmarks typically assume equal relevance across all test items and compute an unweighted sum or average to derive the final score (Wang et al., 2024c; Hendrycks et al., 2021; Rein et al., 2023). To formulate this, for a set of test items Q, we define the

[2]Figure 1 provides a brief overview of benchmark alignment with BENCHALIGN.

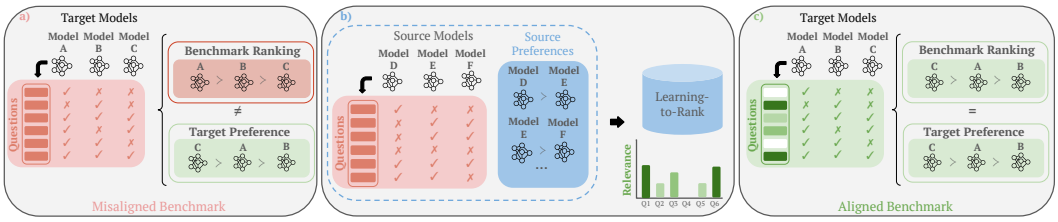


Figure 1: Overview of BENCHALIGN as a diagnostic tool for studying benchmark alignment. Given a benchmark whose ranking of target models is misaligned with a target preference, we use source models and their pairwise preferences to learn question weights via learning-to-rank. The reweighted benchmark is then evaluated to assess whether the induced ranking aligns with the target preference.

performance of a model f_i as a feature vector $\boldsymbol{x}_i \in \mathbb{R}^M$, where the m-th element $x_{i,m} = \mathbf{a}(f_i, q_m)$ represents the instance-level score based on the model's response to q. Rather than assuming uniform relevance, we reformulate the scoring function with a continuous weight vector $\hat{\boldsymbol{w}} \in \mathbb{R}^M$, where each element w_m indicates the learned relevance of test item q_m to the target preference.

Learning-to-Rank. We approach benchmark alignment as a learning-to-rank problem (Li, 2011), which aims to train a model to automatically produce an ordering of a set of objects such that a given utility function is maximized. We focus on the pairwise ranking approach, which learns to predict the relative order of a pair of objects. Pairwise comparisons simplify the evaluation to a binary classification task and have been shown to produce more consistent annotations than absolute rating scales (Thurstone, 1927; Kiritchenko and Mohammad, 2017; Carterette et al., 2008). Furthermore, this approach aligns with the objective of reward modeling used in reinforcement learning from human feedback, which has been shown to effectively model human preferences.[3].

Let $\mathcal{F}$ be the set of models accessible to obtain their output responses on test items Q and their preference ranking $R_T(f_i)$. Given model $f_i \in \mathcal{F}$, we denote $\boldsymbol{x}_i \in \mathbb{R}^M$ by a feature vector containing the instance-level scores $\mathbf{a}(f_i, q)$ computed based on the model's response to all prompts $q \in Q$. We train a pairwise ranking model on inputs $(\boldsymbol{x}_i, \boldsymbol{x}_j)$ for each pair of models (f_i, f_j), where $f_i, f_j \in \mathcal{F}$ and $i \neq j$, minimizing the following pairwise ranking loss (Burges et al., 2005; Wang et al., 2018):

$$\mathcal{L}(\boldsymbol{w}) = \sum_{i=1}^{K} \sum_{j=1}^{K} \mathbb{I}_{r_i < r_j} \log\left(1 + e^{-(s_i - s_j)}\right) \tag{1}$$

where $\mathbb{I}$ is the indicator function, and the term $\log\left(1 + e^{-(s_i - s_j)}\right)$ represents the individual pairwise loss $\mathcal{L}_{pair}(s_i, s_j)$. Here, r_i and r_j denote the target ranks for models f_i and f_j respectively, and $s_i = \boldsymbol{w}^\top \boldsymbol{x}_i$ is the predicted relevance score for model f_i given its performance feature vector $\boldsymbol{x}_i$. The loss function $\mathcal{L}(\boldsymbol{w})$ penalizes misaligned pairs where model f_i is preferred over f_j (i.e., $r_i < r_j$) but the predicted relevance score indicates $s_i < s_j$.

Using BENCHALIGN as a Diagnostic Tool. After minimizing the pairwise ranking loss, we obtain the optimal learned weight vector $\hat{\boldsymbol{w}}$ that captures the relevance of each benchmark question to the target preferences. We use these weights to construct a reweighted benchmark $\hat{\mathcal{D}} = (\hat{Q}, s)$, which applies the reformulated scoring function $s_{\hat{\boldsymbol{w}}}(f_i, Q)$ over the original test items Q. We then evaluate whether the induced ranking $R_{\hat{\mathcal{D}}}$ generalizes to a target set of models $\mathcal{F}'$ unseen during training (i.e., $\mathcal{F} \cap \mathcal{F}' = \varnothing$).

Specifically, we examine whether the learned weights $\hat{\boldsymbol{w}}$ transfer to unseen models, including substantially larger ones. We consider alignment feasible if $R_{\hat{\mathcal{D}}}(f') \approx R_T(f')$ for models in $\mathcal{F}'$, and quantify this using Spearman rank correlation and pairwise ranking accuracy (Section 4.1). We use the degree to which the alignment holds and the data required to achieve it as a measure of how easily existing benchmarks can be aligned.

[3]We provide further details on our algorithm in Appendix C

4 Experiments

We empirically study the feasibility of benchmark alignment and characterize the conditions under which it succeeds. We first describe our experimental setup (Section 4.1) and examine whether aligned benchmarks generalize to unseen models (Section 4.2). We then study the data requirements for alignment, analyzing how many training models (Section 4.3) and benchmark questions (Section 4.4) are needed to achieve meaningful alignment.

4.1 Experimental Setup

We describe the datasets, target preferences, baselines, and metrics used across all experiments.

Benchmarks and Language Models. We primarily use the question-level responses from Open-LLMLeaderboard (Fourrier et al., 2024), which contains responses from 4576 models on 6 standard benchmarks: Big Bench Hard (Suzgun et al., 2022), MMLU Pro (Wang et al., 2024d), MuSR (Sprague et al., 2024), MATH (Hendrycks et al., 2021), GPQA (Rein et al., 2023), and IFEval (Zhou et al., 2023). These benchmarks comprise 21,606 questions across 53 tasks. The dataset spans models from under 1B to over 70B parameters, covering a wide range of model scales.

We keep 4364 models after filtering by availability of responses across all benchmarks[4]. Additionally, we holdout IFEval (541 questions) as this benchmark will be later used for obtaining our target preferences, leaving 52 tasks across 5 benchmarks (21065 questions) available for alignment.

Target Preferences. We simulate target preferences using reward models trained separately on two datasets: (1) Helpsteer (Wang et al., 2023), which consists of 10,459 single-turn prompts with human preferences for helpfulness, coherence, and complexity and (2) UltraFeedback (Cui et al., 2024), which contains over 1 million GPT-4 human preferences for 250k user-assistant conversations across helpfulness, honesty, and verbosity. We focus on helpfulness for Helpsteer and honesty for UltraFeedback, denoting their reward models as Armo, GPT2 and DPA.[5] These reward models score model responses on IFEval to construct a "ground truth" preference ranking over all models.

Baselines. We compare BENCHALIGN against three simple baselines and two benchmark distillation methods: (1) NO ALIGNMENT,[6] which aggregates all benchmarks without reweighting, (2) INDIVIDUAL, which select the task (across 52) with the highest Spearman rank correlation with target preferences as a benchmark (3) RANDOM, which randomly selects one OpenLLMLeaderboard benchmark, (4) METABENCH (Kipnis et al., 2025), which constructs a sparse benchmark by removing questions with redundant information, and (5) TINYBENCHMARKS (Polo et al., 2024), which distills a single benchmark into a smaller set of questions with the most predictive power for model performances. Further implementation details for all methods are described in Appendix E.

Metrics. We use two metrics: 1) the Spearman rank correlation (ρ) measure how closely the predicted rankings match the actual rankings, and 2) *pairwise ranking accuracy* counts how frequently relative model pairs are predicted correctly, as shown in Equation 2, where $\mathcal{P} = \mathcal{F} \times \mathcal{F}$ denotes the set of all model pairs, r_{ij} indicates the ground-truth relative ordering between models f_i and f_j, and $\hat{r}_{ij}$ is the predicted ordering. The indicator function $\mathbb{I}[\cdot]$ outputs 1 if the predicted order is equal to the real order, and 0 otherwise[7].

$$\text{Acc}_{\text{pair}} = \frac{1}{|\mathcal{P}|} \sum_{(i,j)\in\mathcal{P}, i\neq j} \mathbb{I}[\hat{r}_{ij} = r_{ij}] \tag{2}$$

Implementation Details. We implement BENCHALIGN using a linear RankNet model trained with a pairwise ranking loss. We use the Adam optimizer with a learning rate of 1e-4, a margin of separation of 0, and no weight decay unless otherwise indicated. Since the target ranking is given by the reward model's scores evaluated on IFEval, we exclude IFEval from BENCHALIGN's training data[8].

[4]We split models into source and target sets per experiment as described individually below.

[5]All reward models have been used in already published work and are obtained from HuggingFace. Further details are explained in Appendix B. We evaluate our results using 4 additional reward models in Appendix G.3

[6]We use NO ALIGNMENT to refer specifically to the absence of question reweighting, not to imply that other baselines produce preference-aligned rankings.

[7]Margins of error calculated at the 95% confidence unless indicated otherwise

[8]We provide further analysis on our learning-to-rank algorithm and compute resources in Appendix D

Target Models	Method	Helpfulness				Honesty			
		Acc_{pair}		ρ		Acc_{pair}		ρ	
		ArmoRM	GPT2	ArmoRM	GPT2	ArmoRM	DPA	ArmoRM	DPA
70B+	NO ALIGNMENT	0.542 ± 0.010	0.480 ± 0.010	0.154 ± 0.165	0.044 ± 0.251	0.559 ± 0.010	0.567 ± 0.010	0.199 ± 0.164	0.188 ± 0.164
	RANDOM	0.538 ± 0.010	0.468 ± 0.010	0.087 ± 0.161	-0.106 ± 0.166	0.621 ± 0.010	0.605 ± 0.010	0.280 ± 0.145	0.237 ± 0.150
	INDIVIDUAL	0.675 ± 0.009	$\mathbf{0.621 \pm 0.010}$	0.492 ± 0.136	0.306 ± 0.157	0.694 ± 0.009	0.679 ± 0.012	0.536 ± 0.129	0.490 ± 0.136
	METABENCH	0.564 ± 0.007	0.484 ± 0.007	0.200 ± 0.158	-0.028 ± 0.165	0.581 ± 0.007	0.575 ± 0.007	0.246 ± 0.155	0.216 ± 0.157
	TINYBENCHMARKS	0.540 ± 0.010	0.488 ± 0.010	0.147 ± 0.161	-0.024 ± 0.165	0.554 ± 0.010	0.563 ± 0.010	0.187 ± 0.159	0.187 ± 0.159
	BENCHALIGN	$\mathbf{0.778 \pm 0.008}$	0.620 ± 0.010	$\mathbf{0.707 \pm 0.074}$	$\mathbf{0.333 \pm 0.139}$	$\mathbf{0.774 \pm 0.008}$	$\mathbf{0.746 \pm 0.009}$	$\mathbf{0.706 \pm 0.074}$	$\mathbf{0.659 \pm 0.084}$
30B+	NO ALIGNMENT	0.605 ± 0.005	0.477 ± 0.005	0.331 ± 0.114	0.052 ± 0.226	0.618 ± 0.005	0.609 ± 0.005	0.368 ± 0.111	0.319 ± 0.115
	RANDOM	0.551 ± 0.005	0.452 ± 0.005	0.108 ± 0.120	-0.148 ± 0.122	0.650 ± 0.005	0.618 ± 0.005	0.407 ± 0.098	0.317 ± 0.106
	INDIVIDUAL	0.678 ± 0.005	0.606 ± 0.005	0.501 ± 0.098	0.273 ± 0.118	0.692 ± 0.005	0.675 ± 0.005	0.537 ± 0.094	0.480 ± 0.100
	METABENCH	0.606 ± 0.004	0.497 ± 0.004	0.328 ± 0.110	0.007 ± 0.123	0.613 ± 0.004	0.604 ± 0.004	0.346 ± 0.108	0.302 ± 0.112
	TINYBENCHMARKS	0.605 ± 0.005	0.492 ± 0.005	0.323 ± 0.110	-0.016 ± 0.123	0.615 ± 0.005	0.602 ± 0.005	0.354 ± 0.108	0.299 ± 0.112
	BENCHALIGN	$\mathbf{0.765 \pm 0.005}$	$\mathbf{0.637 \pm 0.005}$	$\mathbf{0.710 \pm 0.056}$	$\mathbf{0.387 \pm 0.100}$	$\mathbf{0.773 \pm 0.005}$	$\mathbf{0.763 \pm 0.005}$	$\mathbf{0.730 \pm 0.053}$	$\mathbf{0.713 \pm 0.055}$
13B+	NO ALIGNMENT	0.583 ± 0.002	0.531 ± 0.002	0.245 ± 0.065	0.094 ± 0.067	0.588 ± 0.002	0.577 ± 0.002	0.251 ± 0.064	0.231 ± 0.065
	RANDOM	0.558 ± 0.002	0.537 ± 0.002	0.125 ± 0.066	0.083 ± 0.067	0.603 ± 0.002	0.600 ± 0.002	0.280 ± 0.061	0.272 ± 0.061
	INDIVIDUAL	0.660 ± 0.002	0.608 ± 0.002	0.430 ± 0.057	0.278 ± 0.063	0.661 ± 0.002	0.669 ± 0.002	0.432 ± 0.057	0.451 ± 0.055
	METABENCH	0.584 ± 0.001	0.530 ± 0.001	0.243 ± 0.063	0.089 ± 0.067	0.586 ± 0.001	0.579 ± 0.001	0.248 ± 0.063	0.229 ± 0.064
	TINYBENCHMARKS	0.589 ± 0.002	0.543 ± 0.002	0.259 ± 0.063	0.128 ± 0.066	0.592 ± 0.002	0.583 ± 0.002	0.267 ± 0.063	0.243 ± 0.063
	BENCHALIGN	$\mathbf{0.741 \pm 0.001}$	$\mathbf{0.696 \pm 0.002}$	$\mathbf{0.674 \pm 0.035}$	$\mathbf{0.552 \pm 0.045}$	$\mathbf{0.748 \pm 0.001}$	$\mathbf{0.749 \pm 0.001}$	$\mathbf{0.686 \pm 0.034}$	$\mathbf{0.691 \pm 0.034}$

Table 1: Helpsteer and UltraFeedback results under model scale splits.

4.2 Can aligned benchmarks generalize across model sizes? (RQ 1)

With recent shifts towards larger general-purpose models, we first examine whether aligned benchmarks can generalize to larger unseen models. We therefore test whether benchmark weightings learned from smaller models can accurately predict the preference rankings of much larger models with parameter sizes unobserved during training. We therefore our alignment approach to the baselines on three different source-target splits based on different parameter thresholds (13B, 30B, and 70B). Models above a given threshold are held out for testing, while the remaining smaller models are used for training.[9]

Aligned benchmarks trained on smaller models generalize to larger, unseen models. As shown in Table 1, our proposed method consistently achieves a higher rank correlation (ρ) and pairwise ranking accuracy (Acc_{pair}) than all baselines. METABENCH and TINYBENCHMARKS show much lower and inconsistent correlations than the proposed method, which may be attributed to how both methods are designed to predict benchmark performance rather than human preferences. In fact, we find that they exhibit a similar level of correlations to the NO ALIGNMENT baseline and sometimes even worse than the RANDOM baseline. INDIVIDUAL is the second best baseline in almost all our settings, highlighting the importance of choosing a benchmark related to the target as a proxy of downstream performance. However, an important gap remains between benchmarks and preferences that only benchmark alignment can mitigate effectively.

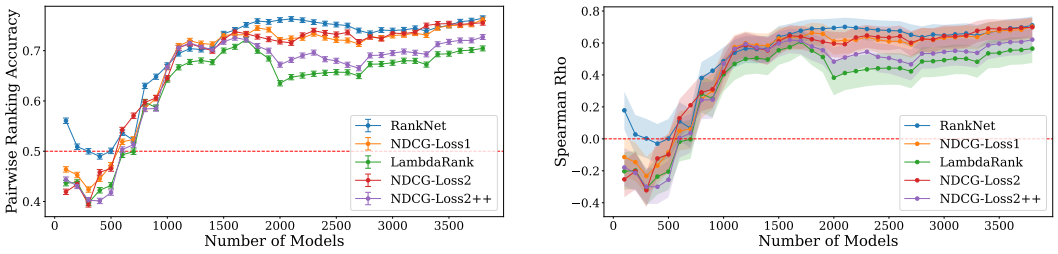
The results in GPT2 further define the current frontier of benchmark alignment. The correlations achieved by NO ALIGNMENT, METABENCH, and TINYBENCHMARKS are close to zero, indicating that overall accuracy across all benchmarks provides no meaningful information about the target preference. INDIVIDUAL and BENCHALIGN still exhibit a positive correlation, but this value is approximately 0.3 and 0.4 points lower than in other preference settings, respectively. This finding suggests that benchmark alignment is harder to achieve when benchmark data and target preferences are poorly correlated.

To summarize, our results show that with sufficient preference signals, aligned benchmarks can generalize across model scales, even when the source data contains only small to medium-sized models. We perform a series of ablations to show the robustness of our results in Appendix G.

4.3 How many training models are needed for benchmark alignment? (RQ 2)

So far, we have applied benchmark alignment using all training models. In practice, collecting large-scale model evaluations and human preference rankings can be computationally and financially prohibitive. Moreover, the choice of the learning-to-rank algorithm may directly impact the number of models required for benchmark alignment, as different methods vary in how efficiently they extract signal from limited pairwise preference data. Therefore, we study how feasible is benchmark alignment under limited data settings. We implement two sets of experiments that use five learning-

[9] Appendix F provides further details on the models used for these experiments.



(a) Pairwise ranking accuracy (Acc_{pair})

(b) Spearman rank coefficient (ρ)

Figure 2: Benchmark alignment under different number of models and learning-to-rank algorithms. Error bars obtained using the number of pairs and sample size from the target set for Acc_{pair} and ρ, respectively. The dotted line represents the results when models are ranked randomly.

to-rank algorithms for alignment[10] on the setup from one of our best results of Section 4.2: ArmoRM (Helpful) as the target preference and 30B+ target models.

Naive benchmark alignment requires a large number of smaller-sized models. We study a naive but realistic scenario in which practitioners incorporate model evaluations as they become available, prioritizing larger, more recent models, without prior knowledge of which model responses are most informative for benchmark alignment. To simulate this setting, we sort the source models by parameter count, train on the smallest 100 models, and progressively expand the training set in increments of 100 as larger models become available.

Figure 2 shows that training with smaller models outputs benchmarks that rank larger models no differently than tossing a coin. This is expected as a considerably low number of parameters (less than 1B parameters) might be insufficient to produce meaningful results on the OpenLLMLeaderboard benchmarks. While including larger models leads to an improvement in the Acc_{pair} and ρ, we find that expanding the model pool with larger-sized models for training does not necessarily lead to more aligned benchmarks. Most implementations show some improvement beyond the first 25% of all OpenLLMLeaderboard models which range between 1 to 7B parameters. However, the gains in rank correlations and pairwise ranking accuracy are marginal after further increments, indicating that smaller models already provide sufficient signals for learning weights that generalize across model scale.

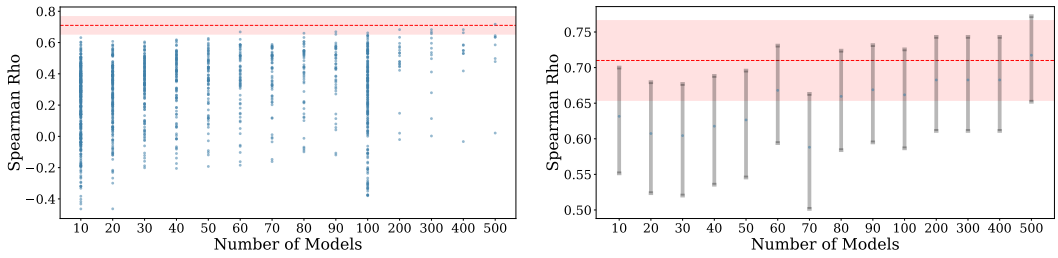
Our results suggest that benchmark alignment needs a large number of small-sized models (less than 7B parameters) to infer the preference ranking of larger-sized models, and that the inclusion of larger models in training does not improve generalization significantly.

Informative subsets enable alignment with as few as ∼20 models. To study whether benchmark alignment can be simplified, we modify our previous setup to identify informative subsets of models ahead of time. Since our previous experiments showed that alignment signals vary across model scales, we first sort the source models by parameter count. We then select a fixed number of neighboring models from the ordered list, align our benchmarks using only those models, and repeat the process by shifting the selected subset across the list. This allows us to identify which subset provides the strongest alignment signals at each subset size.

Figure 3 shows the results for ArmoRM (Helpful), with a target of 30B+ models.[11] We observe a direct relationship between the correlation and the number of models, consistent with our findings from Figure 2, as larger subsets allow for more variety in the model set for performing benchmark alignment. However, a surprising finding is that groups of fewer than 100 models still achieve results within the error margins of using the full training split. Figure 3a shows that most of the benchmarks created with at least 50 models have a correlation skewed upwards. Furthermore, a closer inspection of the best subset of models reveals that we can create aligned benchmarks with far less models. Figure 3b indicates that this requirement can be minimized to 10 models, while showing that the best benchmarks created with similar sized groups are not different from using all models.

[10]The learning-to-rank algorithms are described further in Appendix G.1.

[11]The results for the remaining reward models and target models are in Appendix H.1.



(a) All model subsets

(b) Best-performing model subset

Figure 3: Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point reports the results of the best subset of models by correlation. Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.

Our results suggest that knowing the most informative subset of models could reduce the models needed for alignment to $\sim 0.25\%$ of all models in OpenLLMLeaderboard. To further prove their robustness, we run the same experiment for all reward models and target models from Section 4.2, finding that we can create aligned benchmarks with windows equal or smaller than 20 models for almost all settings (see Appendix H.1). We also study how to select the best subset of models using the model family and generation in Appendix I. However, this is infeasible on these data, suggesting a line of future work.

4.4 How many questions are needed for benchmark alignment? (RQ 3)

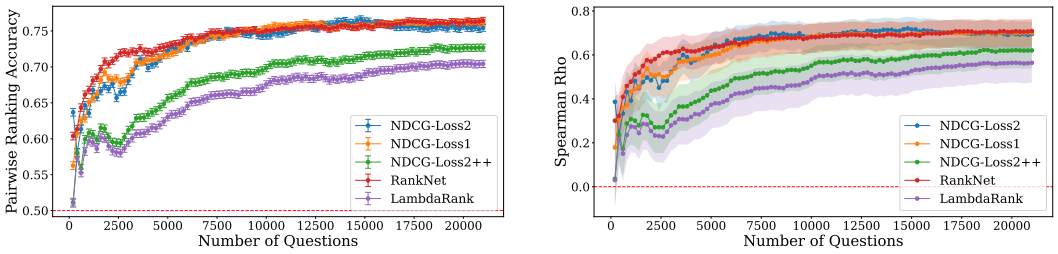
Our results from Section 4.2 suggest that a large number of questions may be required for alignment to be feasible in this exact setting. This would often be prohibitively expensive as it requires significant computing and evaluation resources. However, existing benchmarks may be highly redundant, leading to an artificial overestimate of this number (Kipnis et al., 2025; Polo et al., 2024). We therefore test whether these data requirements can be further minimized on two sets of experiments using the same experimental setup as Section 4.3.

Naive benchmark alignment requires a large fraction of all benchmark questions. Similar to Section 4.3, we design a naive scenario that corresponds to a practitioner without information on the best subset of questions. We first randomly sort the questions and train our alignment implementations with the benchmark responses of all training models on a random subset of 100 questions. Then, we increase the number of questions in steps of 100 until all questions are included.

Figure 4 shows that we can detect signals about model preferences with a limited number of questions, even when using 100 questions across most implementations. Moreover, increasing the number of available questions results in large increases, as shown by the upward trend in both Acc_{pair} and ρ over the first 2500 questions. This result is expected, as augmenting the number of questions increases the probability of the training data capturing a wider variety of questions across different preference levels. This allows the alignment implementations to learn which questions discriminate better according to the target preference. However, the trend reverses when using around 2500 questions for training, except for RANKNET. Among all learning-to-rank algorithms, RANKNET shows more consistent results on both Acc_{pair} and ρ across varying numbers of questions included in training.

Once we increase the number of questions to more than 5000, all benchmark alignment implementations begin to show more steady improvements on both metrics. However, three out of the five implementations, including RANKNET, NDCG-LOSS1, and NDCG-LOSS2, show plateaus in Acc_{pair} and ρ. On the other hand, LAMBDARANK and NDCG-LOSS2++ show lower performance on both metrics, but then steadily improve and ultimately converge to performance comparable to the other implementations. These results indicate that the choice of the learning-to-rank algorithm may affect how the alignment of benchmarks scales with the number of available questions.

Overall, our results show that benchmark alignment needs around 5000 questions ($\sim 25\%$ of all OpenLLMLeaderboard questions) to achieve its best results.



(a) Pairwise ranking accuracy (Acc_{pair})

(b) Spearman rank coefficient (ρ)

Figure 4: Benchmark alignment under different numbers of questions and learning-to-rank algorithms. Error bars obtained using the number of pairs and sample size from the target set for Acc_{pair} and ρ, respectively. The dotted line represents the results when models are ranked randomly.

Benchmark distillation methods can shrink question set requirements. We create a setting where a practitioner knows in advance which are the most predictive questions among all benchmarks by using distillation methods. Specifically, we use METABENCH and TINYBENCHMARKS on all benchmarks jointly to find the set of most predictive questions of benchmark performance and then apply alignment using the responses of all models from the training set on these questions. Table 2 shows the results for ArmoRM (Helpful) and ArmoRM (Honest), with a target of 13B+ models[12]

In all cases, the combination of a benchmark distillation method with benchmark alignment matches or outperforms alignment alone, despite using a reduced set of test items. Particularly, TINYBENCHMARKS achieves the best results. This may be explained by the difference in the number of questions used by the two benchmarks: TINYBENCHMARKS finds 5000 of the most predictive questions, whereas METABENCH uses only 1500. These results not only show that the evaluation costs of aligned benchmarks can be reduced, but also how we may achieve that. We can create aligned benchmarks with as few as 1500 questions, ∼7% of all OpenLLMLeaderboard questions, by relying on the questions most predictive of benchmark performance.

RM	Method	Acc_{pair}	ρ
ArmoRM	NO ALIGNMENT	0.583 ± 0.002	0.245 ± 0.065
	BENCHALIGN	0.741 ± 0.001	0.674 ± 0.035
	METABENCH + BENCHALIGN	0.744 ± 0.001	0.677 ± 0.035
	TINYBENCHMARKS + BENCHALIGN	$\mathbf{0.754 \pm 0.001}$	$\mathbf{0.700 \pm 0.033}$
ArmoRM	NO ALIGNMENT	0.588 ± 0.002	0.251 ± 0.064
	BENCHALIGN	0.748 ± 0.001	0.686 ± 0.034
	METABENCH + BENCHALIGN	0.757 ± 0.001	0.706 ± 0.032
	TINYBENCHMARKS + BENCHALIGN	$\mathbf{0.770 \pm 0.001}$	$\mathbf{0.735 \pm 0.030}$

Table 2: Distillation methods plus benchmark alignment.

We further inspect our findings by replicating this experiment on all reward models and target models from Section 4.2 in Appendix H.2, finding that applying alignment on the subset obtained by a benchmark distillation method produces similar results to applying alignment on all test items.

5 Limitations

There are three key limitations. The first regards reward models as proxies of real world utility. Given the limited availability of large-scale alternatives for capturing human preferences, we rely on reward models trained directly from human feedback (Ouyang et al., 2022; Christiano et al., 2017). However, they may not capture the full heterogeneity of human preferences. We show robustness to such cases on alternative reward models and datasets (Appendix G.3), but incorporating improved proxies of preferences remains an important direction for future work.

The second concerns reliance on ground-truth rankings: alignment requires reliable targets, and if these already exist, the need for a new benchmark is unclear. We acknowledge this limitation, however, this is not our goal. Our approach is designed as a simple tool to study the lack of external validity from existing benchmarks and as a step towards developing novel benchmark alignment methods. Moreover, our results in Sections 4.3 and 4.4 are particularly useful for future works as they show the minimal data requirements for achieving alignment.

Finally, our results show that we need benchmarks that already have a positive correlation with the target preferences for benchmark alignment. We consider this an important finding as it highlights

[12]We choose RankNet as it is the best learning-to-rank algorithm found in Section 4.3.

that practitioners should not aim for aligning arbitrary benchmarks with their targets. Rather, they should aim to align existing benchmarks that are not capturing the utility they were designed to measure.[13]

6 Conclusions

In this paper, we introduce the problem of *benchmark alignment* to address the gap between benchmark performance and its utility for predicting models' downstream preferences. We propose a simple tool and evaluation setup that identifies the limitations for aligning existing benchmarks and show that we can produce new benchmarks that generalize to unseen larger models.

Our work is not intended to replace the development of new benchmarks for tasks where existing evaluations fail to reflect real-world performance. Rather, it aims to study the external validity gap in current evaluations and to establish benchmark alignment as a meaningful direction for future research, while also informing practitioners on its requirements for downstream deployments.

Acknowledgments and Disclosure of Funding

We thank the University of Virginia's Research Computing team for maintaining excellent high-performance computing resources that allowed us to conduct this research.

References

Alaa, A., Hartvigsen, T., Golchini, N., Dutta, S., Dean, F., Raji, I. D., and Zack, T. (2025). Medical Large Language Model Benchmarks Should Prioritize Construct Validity. In *42nd International Conference on Machine Learning, ICML*.

Arora, R., Wei, J., Soskin Hicks, R., Bowman, P., Quiñonero-Candela, J., Tsimpourlas, F., Sharman, M., Shah, M., Vallone, A., Beutel, A., Heidecke, J., and Singhal, K. (2025). HealthBench: Evaluating Large Language Models Towards Improved Human Health.

Bowman, S. R. and Dahl, G. (2021). What will it take to fix benchmarking in natural language understanding? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.

Burges, C., Shaked, T., Renshaw, E., Hamilton, N., and Hullender, G. (2005). Learning to Rank using Gradient Descent. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*.

Busch, F., Hoffmann, L., Rueger, C., van Dijk, E. H., Kader, R., Ortiz-Prado, E., Makowski, M. R., Saba, L., Hadamitzky, M., Kather, J. N., Truhn, D., Cuocolo, R., Adams, L. C., and Bressem, K. K. (2025). Current applications and challenges in large language models for patient care: a systematic review. *Communications Medicine*, 5(1).

Carterette, B., Bennett, P., Chickering, M., and Dumais, S. (2008). Here or there: Preference judgments for relevance. In *Proceedings of ECIR 2008*. Springer.

Christiano, P., Leike, J., Brown, T. B., Martic, M., Legg, S., and Amodei, D. (2017). Deep reinforcement learning from human preferences. In *31st Conference on Neural Information Processing Systems (NIPS 2017)*.

Cui, G., Yuan, L., Ding, N., Yao, G., He, B., Zhu, W., Ni, Y., Xie, G., Xie, R., Lin, Y., Liu, Z., and Sun, M. (2024). UltraFeedback: Boosting Language Models with Scaled AI Feedback. In *Proceedings of the 41st International Conference on Machine Learning*.

Dubois, Y., Li, X., Taori, R., Zhang, T., Gulrajani, I., Ba, J., Guestrin, C., Liang, P., and Hashimoto, T. (2023). Alpacafarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems*.

[13]We discuss our **broader impacts** in Appendix A.

Embretson, S. E. (1998). A cognitive design system approach to generating valid tests: Application to abstract reasoning. *Psychological methods*, 3(3):380.

Fourrier, C., Habib, N., Lozovskaya, A., Szafer, K., and Wolf, T. (2024). Open LLM Leaderboard v2.

Gao, M., Liu, Y., Hu, X., Wan, X., Bragg, J., and Cohan, A. (2025). Re-evaluating automatic LLM system ranking for alignment with human preference. In *Findings of the Association for Computational Linguistics: NAACL 2025*.

Gera, A., Boni, O., Perlitz, Y., Bar-Haim, R., Eden, L., and Yehudai, A. (2025). Justrank: Benchmarking LLM judges for system ranking. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

He, P., Gao, J., and Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. In *35th Conference on Neural Information Processing Systems (NeurIPS 2021) Track on Datasets and Benchmarks.*

Hively, W., Patterson, H. L., and Page, S. H. (1968). A" universe-defined" system of arithmetic achievement tests. *Journal of Educational Measurement*, 5(4):275–290.

Hofmann, V., Heineman, D., Magnusson, I., Lo, K., Dodge, J., Sap, M., Koh, P. W., Wang, C., Hajishirzi, H., and Smith, N. A. (2025). Fluid language model benchmarking. In *Second Conference on Language Modeling*.

Kim, S., Suk, J., Longpre, S., Lin, B. Y., Shin, J., Welleck, S., Neubig, G., Lee, M., Lee, K., and Seo, M. (2024). Prometheus 2: An open source language model specialized in evaluating other language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.

Kipnis, A., Voudouris, K., Buschoff, L. M. S., and Schulz, E. (2025). Metabench: A Sparse Benchmark of Reasoning and Knowledge in Large Language Models. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Kiritchenko, S. and Mohammad, S. (2017). Best-worst scaling more reliable than rating scales: A case study on sentiment intensity annotation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*.

Li, H. (2011). A short introduction to learning to rank. *IEICE TRANSACTIONS on Information and Systems*, 94(10):1854–1862.

Li, J., Sun, S., Yuan, W., Fan, R.-Z., zhao, h., and Liu, P. (2024). Generative judge for evaluating alignment. In *International Conference on Learning Representations*.

Liao, T., Taori, R., Raji, D., and Schmidt, L. (2021). Are we learning yet? a meta review of evaluation failures across machine learning. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.

Liu, Y., Moosavi, N. S., and Lin, C. (2024). LLMs as narcissistic evaluators: When ego inflates evaluation scores. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang Sandhini Agarwal Katarina Slama Alex Ray John Schulman Jacob Hilton Fraser Kelton Luke Miller Maddie Simens Amanda Askell, C., Welinder Paul Christiano, P., Leike, J., and Lowe, R. (2022). Training language models to follow instructions with human feedback.

Pecher, B., Srba, I., and Bielikova, M. (2025). Comparing Specialised Small and General Large Language Models on Text Classification: 100 Labelled Samples to Achieve Break-Even Performance. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 165–184.

Polo, F. M., Weber, L., Choshen, L., Sun, Y., Xu, G., and Yurochkin, M. (2024). tinyBenchmarks: evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, Vienna, Austria.

Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. (2023). GPQA: A Graduate-Level Google-Proof Q&A Benchmark.

Reuel, A., Hardy, A., Smith, C., Lamparth, M., Hardy, M., and Kochenderfer, M. J. (2024). Betterbench: Assessing AI benchmarks, uncovering issues, and establishing best practices. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Rodriguez, P., Barrow, J., Hoyle, A., Lalor, J. P., Jia, R., and Boyd-Graber, J. (2021). Evaluation examples are not equally informative: How should that change NLP leaderboards? In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*.

Salaudeen, O. and Hardt, M. (2025). ImageNot: A contrast with ImageNet preserves model rankings.

Saxon, M., Holtzman, A., West, P., Wang, W. Y., and Saphra, N. (2024). Benchmarks as microscopes: A call for model metrology. In *First Conference on Language Modeling*.

Sprague, Z., Ye, X., Bostrom, K., Chaudhuri, S., and Durrett, G. (2024). MuSR: Testing the Limits of Chain-of-thought with Multistep Soft Reasoning. In *The Twelfth International Conference on Learning Representations*.

Subramonian, A., Yuan, X., Daumé III, H., and Blodgett, S. L. (2023). It takes two to tango: Navigating conceptualizations of NLP tasks and measurements of performance. In *Findings of the Association for Computational Linguistics: ACL 2023*.

Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q. V., Chi, E. H., Zhou, D., and Wei, J. (2022). Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them.

Thurstone, L. (1927). A law of comparative judgment. *Psychological Review*, 34:273–86.

Vania, C., Htut, P. M., Huang, W., Mungra, D., Pang, R. Y., Phang, J., Liu, H., Cho, K., and Bowman, S. R. (2021). Comparing test sets with item response theory. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*.

Vivek, R., Ethayarajh, K., Yang, D., and Kiela, D. (2024). Anchor points: Benchmarking models with much fewer examples. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Wang, H., Xiong, W., Xie, T., Zhao, H., and Zhang, T. (2024a). Interpretable Preferences via Multi-Objective Reward Modeling and Mixture-of-Experts. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10582–10592.

Wang, H. W., Lin, Y. L., Xiong, W. X., Yang, R., Diao, S., Qiu, S., Zhao, H., and Zhang, T. (2024b). Arithmetic Control of LLMs for Diverse User Preferences: Directional Preference Alignment with Multi-Objective Rewards. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, volume 1, pages 8642–8655.

Wang, P., Li, L., Chen, L., Cai, Z., Zhu, D., Lin, B., Cao, Y., Kong, L., Liu, Q., Liu, T., and Sui, Z. (2024c). Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Wang, X., Li, C., Golbandi, N., Bendersky, M., and Najork, M. (2018). The LambdaLoss framework for ranking metric optimization. In *International Conference on Information and Knowledge Management, Proceedings*, pages 1313–1322. Association for Computing Machinery.

Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., and Chen, W. (2024d). MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In *38th Conference on Neural Information Processing Systems*.

Wang, Z., Dong, Y., Zeng, J., Adams, V., Sreedhar, M. N., Egert, D., Delalleau, O., Scowcroft, J. P., Kant, N., Swope, A., and Kuchaiev, O. (2023). HelpSteer: Multi-attribute Helpfulness Dataset for SteerLM.

Yang, R., Pan, X., Luo, F., Qiu, S., Zhong, H., Yu, D., and Chen, J. (2024). Rewards-in-Context: Multi-objective Alignment of Foundation Models with Dynamic Preference Adjustment. In *Proceedings of the 41 st International Conference on Machine Learning*.

Zhang, G., Dominguez-Olmedo, R., and Hardt, M. (2025a). Train-before-Test Harmonizes Language Model Rankings.

Zhang, G., Dorner, F. E., and Hardt, M. (2025b). How benchmark prediction from fewer data misses the mark. In *NeurIPS 2025 Workshop on Evaluating the Evolving LLM Lifecycle: Benchmarks, Emergent Abilities, and Scaling*.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*.

Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. (2023). Instruction-Following Evaluation for Large Language Models.

Zhou, Y., Nderitu, P., Hui, J., Goh, L., Engelmann, J., Wagner, S. K., Ran, A., Jiang, H., Zou, K., Srinivasan, S., Kim, H., Ninomiya, T., Wang, Z., Yang, G. D., Ruffell, E., Williamson, D., Santos, R., Somfai, G. M., Cheung, C. Y., Wong, T. Y., Alexander, D. C., Tham, Y. C., and Keane, P. A. (2025). Generalist versus Specialist Vision Foundation Models for Ocular Disease and Oculomics.

A Broader Impacts

This paper analyses whether introduces the problem of benchmark alignment using limited benchmark data and rankings derived from pairwise preferences. Our goal is to advance the field of machine learning by creating new benchmarks from existing ones that better reflect a set of target preferences.

We consider that model selection for downstream deployment is an application of our work with potential societal consequences. Aligned benchmarks might be used on high-stakes settings such as healthcare, where rankings of language models in real-world clinical tasks are scarce. This also poses the risk of misuse as using the wrong subset of models can negatively impact on a patients health.

In addition, benchmark alignment could be used as a tool for benchmark diagnosis as our work offers a framework to determine whether existing benchmarks are aligned with real-world utility based on the pairwise ranking accuracy and the spearman rank correlation.

Finally, aligned benchmarks could be used to construct leaderboards of models that have yet to be tested for our target ranking. Our results suggest we can build generalizable benchmarks for larger models relying solely on small and medium sized model data.

B Reward Models

The main preferences we consider are Helfulness and Honesty as they are the standard in the model alignment literature to evaluate whether generations are consistent with human preferences.

We use three reward models from HuggingFace trained on Helpsteer-Helpfulness (Wang et al., 2023) and UltraFeedback-Honesty (Cui et al., 2024) as described in the following list:

- ArmoRM (Helpful): ArmoRM, Helpsteer-Helpfulness dimension (Wang et al., 2024a)
- GPT2: GPT2-Large-helpful-reward-model (Yang et al., 2024)
- ArmoRM (Honest): ArmoRM, UltraFeedback-Honesty dimension (Wang et al., 2024a)
- DPA: RewardModel-Mistral-7b-for-DPA-v1 (Wang et al., 2024b)

ArmoRM (Wang et al., 2024a) is a reward model trained to output a vector of rewards, with each dimension corresponding to a category of human values datasets such as HelpSteer and UltraFeedback. We used the Helpsteer-Helpfulness and UltraFeedback-Honesty as shown above.

Six models used to train HelpSteer and UltraFeedback were also present in our OpenLLM-Leaderboard dataset and were therefore excluded from our experiments. These models are: “wizardlmteam/wizardlm-13b-v1.2," “meta-llama/llama-2-7b-hf," “meta-llama/llama-2-13b-hf," “meta-llama/llama-2-70b-chat-hf," “tiiuae/falcon-40b-instruct," and “eleutherai/pythia-12b."

To proof the robustness of our results, we also consider the Coherence, Complexity and Verbosity preferences from ArmoRM in Tables 3 and 4.

C BenchAlign Expanded Description

C.1 A Framework for Benchmark Construction

Similar to educational testing, previous work has argued that not all test items are equally informative for model evaluation (Rodriguez et al., 2021). This idea has led to methods for improving benchmark predictability (Zhang et al., 2025b). The most prominent approach is benchmark distillation (Vivek et al., 2024; Polo et al., 2024; Kipnis et al., 2025), which constructs new benchmarks by selecting a smaller subset of test items that preserves the predictive power of model performances on the benchmark. This approach is equivalent to applying a binary weight vector $\boldsymbol{w} \in \{0,1\}^M$, where $w_m = 1$ for selected items and $w_m = 0$ for the rest.

While benchmark distillation methods provide a more efficient way to evaluate model performance, recent work by Rodriguez et al. (2021) shows that the success of these methods primarily relies on *model similarity*: they do well on similar models but fail to generalize when source models (used for constructing new benchmarks) and target models are drawn from different distributions.

Target Models	Method	Coherence		Complexity		Verbosity	
		Acc_{pair}	ρ	Acc_{pair}	ρ	Acc_{pair}	ρ
70B+	NO ALIGNMENT	0.519 ± 0.010	0.080 ± 0.162	0.569 ± 0.010	0.205 ± 0.153	0.536 ± 0.010	0.104 ± 0.160
	RANDOM	0.576 ± 0.010	0.210 ± 0.152	0.585 ± 0.010	0.223 ± 0.151	0.552 ± 0.010	0.103 ± 0.160
	INDIVIDUAL	0.653 ± 0.009	0.426 ± 0.126	0.674 ± 0.009	0.448 ± 0.123	$\mathbf{0.638 \pm 0.009}$	$\mathbf{0.355 \pm 0.136}$
	METABENCH	0.542 ± 0.007	0.127 ± 0.162	0.587 ± 0.007	0.251 ± 0.155	0.555 ± 0.007	0.149 ± 0.161
	TINYBENCHMARKS	0.517 ± 0.007	0.076 ± 0.164	0.562 ± 0.007	0.180 ± 0.160	0.527 ± 0.007	0.080 ± 0.164
	BENCHALIGN	$\mathbf{0.765 \pm 0.008}$	$\mathbf{0.681 \pm 0.079}$	$\mathbf{0.701 \pm 0.009}$	$\mathbf{0.540 \pm 0.107}$	0.482 ± 0.010	-0.037 ± 0.166
30B+	NO ALIGNMENT	0.586 ± 0.005	0.272 ± 0.110	0.609 ± 0.005	0.319 ± 0.106	0.541 ± 0.005	0.119 ± 0.119
	RANDOM	0.611 ± 0.005	0.320 ± 0.106	0.608 ± 0.005	0.295 ± 0.108	0.547 ± 0.005	0.113 ± 0.120
	INDIVIDUAL	0.670 ± 0.005	0.462 ± 0.091	0.666 ± 0.005	0.445 ± 0.093	$\mathbf{0.594 \pm 0.005}$	$\mathbf{0.263 \pm 0.111}$
	METABENCH	0.592 ± 0.004	0.282 ± 0.113	0.590 ± 0.004	0.269 ± 0.114	0.556 ± 0.004	0.158 ± 0.120
	TINYBENCHMARKS	0.588 ± 0.004	0.269 ± 0.114	0.605 ± 0.004	0.305 ± 0.112	0.542 ± 0.004	0.125 ± 0.121
	BENCHALIGN	$\mathbf{0.763 \pm 0.005}$	$\mathbf{0.707 \pm 0.057}$	$\mathbf{0.718 \pm 0.005}$	$\mathbf{0.600 \pm 0.073}$	0.573 ± 0.005	0.222 ± 0.114
13B+	NO ALIGNMENT	0.574 ± 0.002	0.215 ± 0.063	0.582 ± 0.002	0.239 ± 0.063	0.570 ± 0.002	0.207 ± 0.064
	RANDOM	0.636 ± 0.002	0.357 ± 0.057	0.572 ± 0.002	0.209 ± 0.064	0.543 ± 0.002	0.088 ± 0.067
	INDIVIDUAL	0.655 ± 0.002	0.420 ± 0.054	0.650 ± 0.002	0.403 ± 0.055	0.608 ± 0.002	0.311 ± 0.060
	METABENCH	0.573 ± 0.001	0.211 ± 0.064	0.582 ± 0.001	0.238 ± 0.064	0.572 ± 0.001	0.215 ± 0.064
	TINYBENCHMARKS	0.581 ± 0.001	0.237 ± 0.064	0.584 ± 0.001	0.245 ± 0.063	0.553 ± 0.001	0.155 ± 0.066
	BENCHALIGN	$\mathbf{0.745 \pm 0.001}$	$\mathbf{0.683 \pm 0.034}$	$\mathbf{0.726 \pm 0.001}$	$\mathbf{0.622 \pm 0.040}$	$\mathbf{0.614 \pm 0.002}$	$\mathbf{0.330 \pm 0.059}$

Table 3: Alignment with other preference dimensions under model scale splits.

Target Models	Method	Coherence		Complexity		Verbosity	
		Acc_{pair}	ρ	Acc_{pair}	ρ	Acc_{pair}	ρ
2%	NO ALIGNMENT	0.711 ± 0.015	0.583 ± 0.124	0.697 ± 0.015	0.562 ± 0.129	0.575 ± 0.016	0.222 ± 0.191
	RANDOM	0.661 ± 0.015	0.468 ± 0.150	0.699 ± 0.015	0.561 ± 0.129	0.543 ± 0.016	0.108 ± 0.204
	INDIVIDUAL	0.724 ± 0.014	0.608 ± 0.118	0.707 ± 0.015	0.576 ± 0.125	0.712 ± 0.015	0.425 ± 0.158
	METABENCH	0.717 ± 0.010	0.593 ± 0.139	0.702 ± 0.010	0.572 ± 0.144	0.573 ± 0.011	0.219 ± 0.201
	TINYBENCHMARKS	0.704 ± 0.010	0.570 ± 0.144	0.684 ± 0.011	0.543 ± 0.151	0.576 ± 0.011	0.231 ± 0.200
	BENCHALIGN	$\mathbf{0.864 \pm 0.011}$	$\mathbf{0.888 \pm 0.037}$	$\mathbf{0.839 \pm 0.012}$	$\mathbf{0.868 \pm 0.044}$	$\mathbf{0.745 \pm 0.014}$	$\mathbf{0.637 \pm 0.110}$
5%	NO ALIGNMENT	0.696 ± 0.006	0.548 ± 0.087	0.703 ± 0.006	0.569 ± 0.084	0.532 ± 0.006	0.104 ± 0.130
	RANDOM	0.660 ± 0.006	0.457 ± 0.099	0.703 ± 0.006	0.566 ± 0.084	0.508 ± 0.006	0.005 ± 0.133
	INDIVIDUAL	0.714 ± 0.006	0.584 ± 0.081	0.718 ± 0.006	0.585 ± 0.081	0.643 ± 0.006	0.256 ± 0.120
	METABENCH	0.697 ± 0.004	0.549 ± 0.093	0.703 ± 0.004	0.568 ± 0.090	0.534 ± 0.004	0.104 ± 0.131
	TINYBENCHMARKS	0.691 ± 0.004	0.539 ± 0.095	0.697 ± 0.004	0.561 ± 0.092	0.539 ± 0.004	0.124 ± 0.131
	BENCHALIGN	$\mathbf{0.847 \pm 0.005}$	$\mathbf{0.862 \pm 0.031}$	$\mathbf{0.820 \pm 0.005}$	$\mathbf{0.829 \pm 0.037}$	$\mathbf{0.719 \pm 0.006}$	$\mathbf{0.588 \pm 0.081}$
10%	NO ALIGNMENT	0.664 ± 0.003	0.466 ± 0.070	0.678 ± 0.003	0.501 ± 0.067	0.525 ± 0.003	0.077 ± 0.093
	RANDOM	0.639 ± 0.003	0.396 ± 0.076	0.678 ± 0.003	0.495 ± 0.068	0.500 ± 0.003	-0.024 ± 0.094
	INDIVIDUAL	0.691 ± 0.003	0.520 ± 0.065	0.688 ± 0.003	0.512 ± 0.066	0.584 ± 0.003	0.233 ± 0.087
	METABENCH	0.660 ± 0.002	0.457 ± 0.074	0.674 ± 0.002	0.492 ± 0.071	0.524 ± 0.002	0.072 ± 0.093
	TINYBENCHMARKS	0.661 ± 0.002	0.457 ± 0.074	0.672 ± 0.002	0.489 ± 0.072	0.530 ± 0.002	0.092 ± 0.093
	BENCHALIGN	$\mathbf{0.850 \pm 0.002}$	$\mathbf{0.866 \pm 0.022}$	$\mathbf{0.832 \pm 0.002}$	$\mathbf{0.845 \pm 0.025}$	$\mathbf{0.767 \pm 0.003}$	$\mathbf{0.700 \pm 0.045}$

Table 4: Alignment with other preference dimensions under arbitrary model sets

Rather than assuming uniform relevance or applying binary weights to test items, we propose reformulating the benchmark scoring function with a continuous weight vector $\boldsymbol{w} \in \mathbb{R}$. Each element w_m indicates the learned relevance of test item q_m to the target preference. The new scoring function $s_{\boldsymbol{w}}$ computes the system-level score for model f_i as the dot product of the normalized weights and the model's performance vector $\boldsymbol{x}_i$:

$$s_{\boldsymbol{w}}(f_i, Q) = \sum_{m=1}^{M} \tilde{w}_m x_{i,m} = \tilde{\boldsymbol{w}}^{\mathsf{T}} \boldsymbol{x}_i \tag{3}$$

where $\tilde{\boldsymbol{w}}$ is a normalized weight vector, computed as $w_m / \sum_{j=1}^{M} w_j$. While the instance-level score $x_{i,m}$ varies by tasks, we assume a binary setting where $x_{i,m} \in \{0, 1\}$, which includes common metrics like accuracy. If $x_{i,m}$ is determined by correctness, the scoring function s corresponds to weighted accuracy. The system-level scores produced by s are used to determine the final model ranking. By optimizing the continuous weight vector $\tilde{\boldsymbol{w}}$, we construct an aligned benchmark $\hat{\mathcal{D}}$ in which test items are scaled by their utility for predicting the target ranking.

C.2 Training BenchAlign

Algorithm 1 summarizes our approach for training the BENCHALIGN model.

Algorithm 1 BenchAlign

Input: $X = [\boldsymbol{x}_1, ..., \boldsymbol{x}_K]$ feature vectors of all models
Input: $r = [r_1, ..., r_K]$ target ranking R_T for all models
Input: $\mathcal{L}_{pair}$ loss function
Input: b batch size
Output: $\boldsymbol{w}$ learned weight vector
Initialize $\boldsymbol{w} \leftarrow$ RANDOM()
ordered pair generation
$P \leftarrow \emptyset$ {set of ordered pairs}
for $i = 1$ to K **do**
  for $j = 1$ to K **do**
    if $r_i < r_j$ **then**
      $P \leftarrow P \cup \{(i, j)\}$
    end if
  end for
end for
learning-to-rank
for each epoch **do**
  Shuffle P
  for each batch $B \subset P$ of size b **do**
    $L \leftarrow 0$
    for all $(i, j) \in B$ **do**
      Compute scores $s_i = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_i$ and $s_j = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_j$
      $L \leftarrow L + \mathcal{L}_{pair}(s_i, s_j)$
    end for
    $L \leftarrow L/|B|$
    Update $\boldsymbol{w}$ by minimizing L
  end for
end for
return $\boldsymbol{w}$

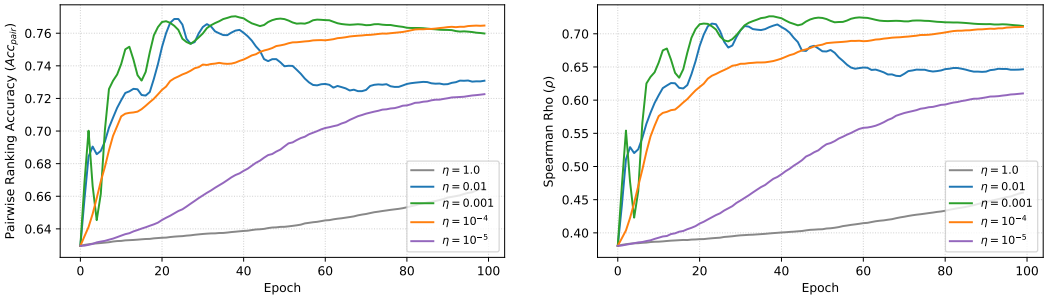
D Learning-to-rank Algorithm for Benchmark Alignment

D.1 Number of Layers

According to our problem definition, benchmark alignment can be performed in different ways. One of them could be designing a bigger neural network than the one used for our experiments, This architecture is a deliberate choice aimed to create better and interpretable evaluations using a simple approach. As shown in Section 4.2, this architecture is sufficient to achieve generalization, and provides a starting point towards more complex methods. Multilayer models, e.g. reward models, would likely lead to better predictions results, but cost tractability and interpretation. We delve deeper on interpretability in Appendix D.4.

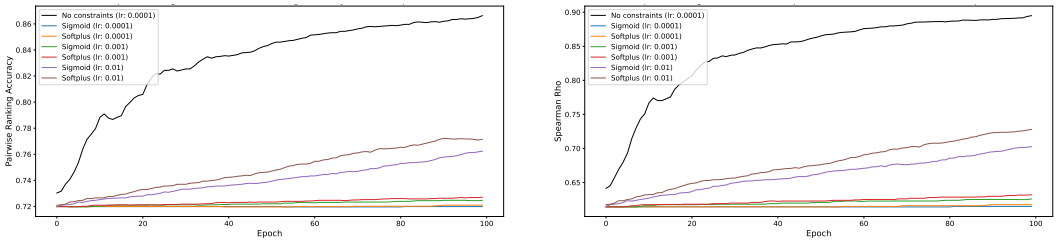
D.2 Hyperparameter Analysis

We complement the architectural analysis above with a hyperparameter study focused on the learning rate, the only optimization hyperparameter that materially affected BENCHALIGN performance in our experiments. Owing to the simplicity of the model, the optimization space is correspondingly small. We evaluate $\eta \in 1, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}$ on the ArmoRM (Helpful) preference using the 30B+ split, training each configuration for 100 epochs and tracking test-set Acc_{pair} and ρ throughout training. Figure 5 summarizes the results. Both extremes of the sweep behave as expected. With $\eta = 1$, optimization is unstable and the model quickly plateaus, while $\eta = 10^{-5}$ yields updates that are too small for meaningful progress within 100 epochs. The intermediate regime is comparatively robust: both $\eta = 10^{-3}$ and $\eta = 10^{-4}$ converge to roughly $Acc_{pair} \approx 0.76$ and $\rho \approx 0.71$. The primary difference between the two lies in convergence speed, with $\eta = 10^{-3}$ reaching its plateau after roughly 30 epochs and $\eta = 10^{-4}$ requiring closer to 60. Although $\eta = 10^{-2}$ converges rapidly, it stabilizes at slightly worse final metrics, suggesting mild overshooting.



(a) Pairwise ranking accuracy (Acc_{pair}) (b) Spearman rank coefficient (ρ)

Figure 5: Effect of learning rate on BENCHALIGN test-time performance over 100 epochs.



(a) Pairwise ranking accuracy (Acc_{pair}) (b) Spearman rank coefficient (ρ)

Figure 7: Comparison of ranking performance of unconstrained vs. constrained weights over the first 100 training epochs.

D.3 Constraining Items

We don't impose non-negativity constraints as they remove useful signals from questions that are used for ranking a model lower according to the target preferences. To prove that these constraints would limit alignment, we replicate our ArmoRM (Helpful) experiment on the 2% target set from Table 11 with 8 different configurations that impose a non-negativity constraint as shown in Figure 7

D.4 Interpretability of Benchmark Alignment

We design a deliberately challenging setting that forces alignment to recover signal from latent relationships across available benchmark tasks. Specifically, we chose the ranking from MMLU-Pro Physics as our target preferences, using the 13B+ models as target[14] and six candidate tasks where only one, Math Geometry Hard, has a direct but mild relationship with the target[15]. The remaining candidates are the five tasks with the lowest target correlation on the train split. The benchmarks and number of questions are indicated in Table 5

[14] In this setup, we removed the ties from the target as RankNet based approaches are designed for a strict preference ordering. Hence, giving the tied models an arbitrary order would be a source of noise against BENCHALIGN

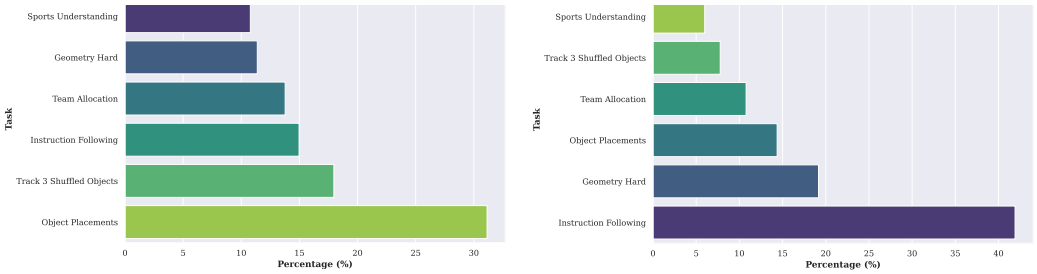
[15] Math Geometry Hard has the lowest train correlation with the target preferences among all STEM benchmark tasks

Table 5: Benchmarks used for MMLU-Pro Physics target preferences

Benchmark	Task	Number of Questions
IFEval	Instruction Following	541
MUSR	Object Placements	256
BBH	Sports Understanding	250
BBH	Tracking Shuffled Three Objects	250
MUSR	Team Allocation	250
MATH	Geometry Hard	132
Total		1679

First, we analyze how benchmark alignment determines the least and most relevant tasks for benchmark alignment. As explained in Section 3.2, BENCHALIGN learns weights that capture the relevance of every question with respect to the target preferences. These weights range between -1 and 1 (Burges et al., 2005), such that questions with weights close to the extremes are the most informative for discriminating between models and determining their ranking. In contrast, questions with weights close to 0 have little to no influence. For simplicity, we take the absolute value of these weights to focus on the least and most relevant questions.

Figure 8 shows the distribution of tasks among questions in the top and bottom 10% of absolute weights. A surprising finding is that questions from all six benchmarks appear in both the top and bottom weight groups. While it would be expected to find questions from a task related to the target such as Geometry to be the dominant group within the top 10 %, we find that Instruction Following, an unrelated benchmark, is ahead by a large margin. These results suggest that BENCHALIGN draws signal from questions across all available benchmarks, rather than relying on topic relationship between candidate benchmarks and the target preferences.



(a) Top 10%

(b) Bottom 10%

Figure 8: Topic distribution of questions in the top 10% and bottom 10% by absolute weight learned by BENCHALIGN.

To further investigate this finding, we also perform a qualitative assessment of the least and most relevant questions in Table 6. We do not observe an obvious semantic pattern indicating that highly weighted questions are more closely related to physics than low-weight questions. For example, two of the questions with the highest absolute weights involve narrative reasoning about the location of an object in a story, while another involves writing a rap song. These examples appear unrelated to physics. In contrast, several of the questions with near-zero weights are drawn from the Math Geometry Hard benchmark, which is arguably more semantically related to physics. These observations suggest that the learned weights do not capture surface-level semantic similarity between benchmarks and the target task.

We next analyze how benchmark alignment changes the task-level correlations with the target preferences. Figure 9 provides a clearer picture on how our approach performs alignment. The largest improvements occur for Sports Understanding and Tracking 3 Shuffled Objects, while Geometry Hard remains the most correlated benchmark overall, with its correlation increasing by only 0.01. At the task level, BENCHALIGN increases the target correlation of the lowest correlated tasks while keeping the correlation of the remaining ones around their same value. However, a more interesting finding is observed in two-task combinations as they show substantially larger improvements. Particularly,

Table 6: Examples of questions with the highest and lowest absolute weights learned by BENCHALIGN.

$\|w\|$	Benchmark	Task	Question
0.0118	MUSR	Object Placements	In the quiet afternoon, Madison was preparing for an important parent–teacher conference when she realized her gradebook was missing. While searching the classroom with Rachel and the school custodian Alex, several objects were moved around the room... **Question:** Which location is the most likely place Madison would look to find the gradebook?
0.0117	IFEval	Instruction Following	Generate a list of 100 random names. Make sure that no name is repeated and every name is unique. All letters in your entire response should be capitalized. Italicize 5 of your favorite names.
0.0117	MUSR	Object Placements	Charlie had just finished his novel and stored the manuscript in the cupboard while working with his assistant Lisa and roommate Matthew. As they moved around the room during various tasks, the manuscript and other objects were relocated... **Question:** Which location is the most likely place Lisa would look to find the manuscript?
0.0117	IFEval	Instruction Following	Can you write a rap that doesn't include the keywords "Yo", "check", and "peace"?
0.0117	MATH	Geometry Hard	A 10-cm stick has a mark at each centimeter. By breaking the stick at two of these nine marks at random, the stick is split into three pieces, each of integer length. What is the probability that the three lengths could be the three side lengths of a triangle? Express your answer as a common fraction.
0.0	MATH	Geometry Hard	In acute triangle ABC, altitudes AD, BE, and CF intersect at the orthocenter H. If $BD = 5$, $CD = 9$, and $CE = 42/5$, then find the length of HE.
0.0	MUSR	Object Placements	Oliver was preparing for a music school audition while practicing violin with support from Peter and supervision from Evelyn. As they moved around the room during practice, instruments and music sheets were relocated between the case, stand, and piano bench... **Question:** Which location is the most likely place Peter would look to find the violin bow?
0.0	MATH	Geometry Hard	In triangle ABC, $AB = 17$, $AC = 8$, and $BC = 15$. Let D be the foot of the altitude from C to AB. Find the area of triangle ACD.
0.0	MATH	Geometry Hard	Let $ABCD$ be a regular tetrahedron with side length 2. The plane parallel to edges AB and CD and lying halfway between them cuts $ABCD$ into two pieces. Find the surface area of one of these pieces.
0.0	MATH	Geometry Hard	In $\triangle ABC$ we have $AB = 7$, $AC = 8$, and $BC = 9$. Point D is on the circumscribed circle of the triangle so that $\overline{AD}$ bisects $\angle BAC$. What is the value of AD/CD?

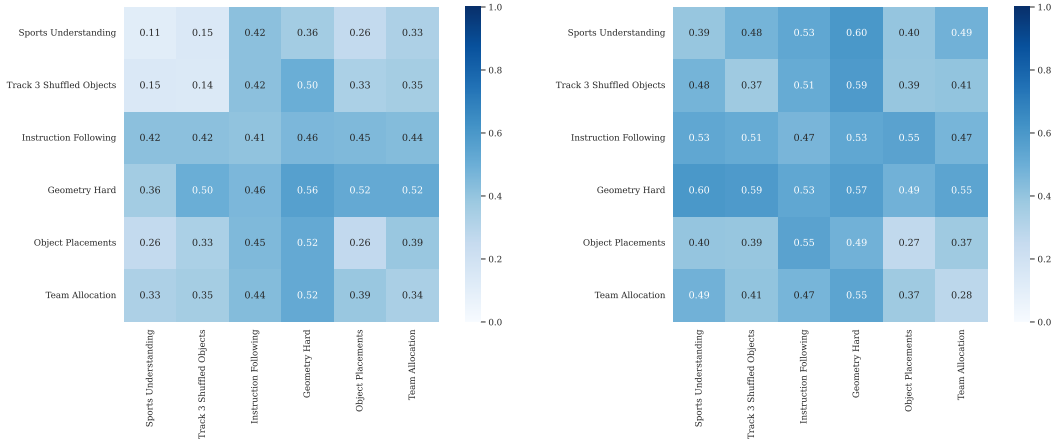
the correlation of Sports Understanding and Tracking 3 Shuffled Objects combined increases from approximately 0.15 to 0.48. In most combinations, correlation increases after applying BENCHALIGN, while the few combinations that decrease do so by at most 0.06. These results suggest that the weights learned by BENCHALIGN amplify complementary signals across benchmarks rather than simply reinforcing the most individually correlated benchmarks or questions.

An important implication is that the learned weights do not merely reflect how strongly each benchmark task correlates with the target ranking in isolation. Instead, they encode how individual questions contribute to ranking models according to the latent capabilities they measure, both individually and in interaction with questions from other benchmarks. Benchmark alignment therefore identifies cross-task relationships that may not be evident from benchmark-level correlations alone.

In addition, we designed an experiment where the target is a MMLU Physics Pro and the candidate benchmarks are non-STEM and have the lowest correlations to the target (Figure 11). We measured the individual and joint correlations before and after alignment, ultimately finding that alignment recovers more meaningful rankings by finding latent relationships between benchmarks as shown by the correlation obtained when combining two tasks.

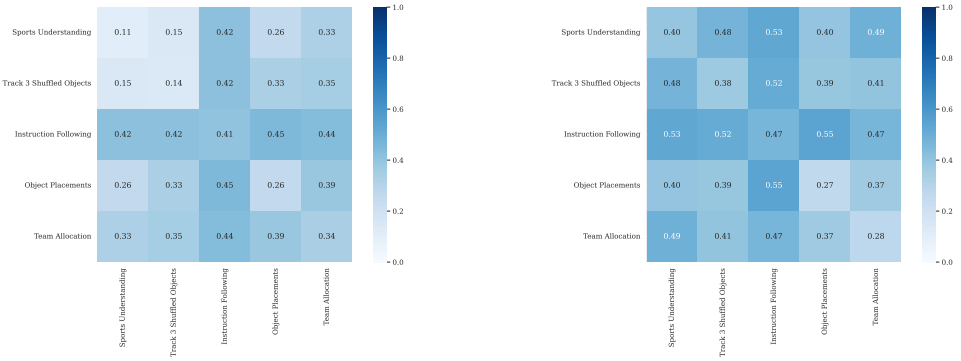
We replicated the same experiment as above but considering candidate benchmarks that were STEM and have the highest correlations to the target (Figure 13). The individual and joint correlations show that BenchAlign marginally reduces the relevance of these benchmarks. This is expected as performing benchmark alignment on evaluations already aligned with the target introduces noise in the signals captured by their test items.

Finally, we determine whether the question sets found by alignment are stable for a set of preferences across different seeds and not randomly determined by repeating the 5% and 10% experiments on RM_1^{Judge} from Appendix G.3 (Preference Heterogeneity) using 5 different seeds to determine whether the high-weight questions from the initial results are stable. The target metric was the overlap (%) of the top 10% questions with the highest weights across seeds (Table 7). Our results show that



(a) Before BENCHALIGN (b) After BENCHALIGN

Figure 9: Target correlation between benchmark-induced model rankings and target ranking. The diagonal shows the correlation obtained when using each benchmark individually. Off-diagonal cells show the correlation obtained when using two joint benchmarks.



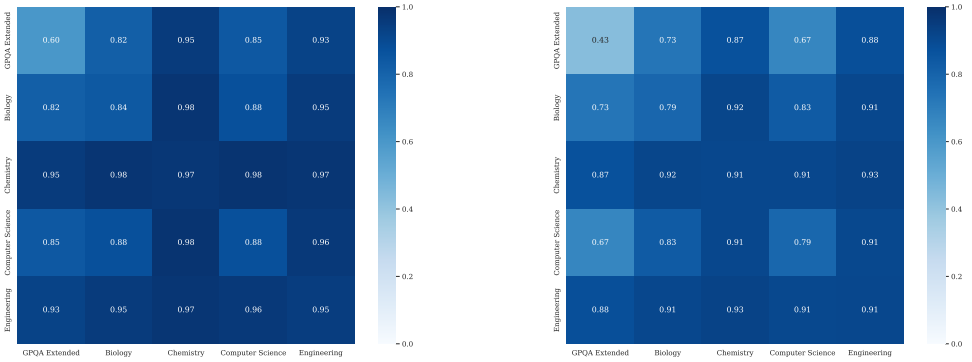
(a) Before BENCHALIGN (b) After BENCHALIGN

Figure 11: Substantially Different Benchmarks. Target correlation between benchmark-induced model rankings and target ranking. The diagonal shows the correlation obtained when using each benchmark individually. Off-diagonal cells show the correlation obtained when using two joint benchmarks.

BenchAlign finds a consistent set of questions for the same preferences regardless of and varying conditions.

D.5 Compute Resources and Runtime Analysis

We run our experiments on the HPC of the University of Virginia using the GPUs provided by the system based on availability. To assess the specific compute requirements and runtime analysis, we replicated our experiments from Section 4.3 from Figure 2 with different number of epochs, learning rates and number of models on a NVIDIA RTX A6000 with 48 GBs of VRAM. We show our results in Table 8. As observed, the biggest factor influencing runtime is the number of epochs, while the number of models and learning rate have little effect on training time.



(a) Before BENCHALIGN (b) After BENCHALIGN

Figure 13: Highly Relevant Benchmarks. Target correlation between benchmark-induced model rankings and target ranking. The diagonal shows the correlation obtained when using each benchmark individually. Off-diagonal cells show the correlation obtained when using two joint benchmarks.

Table 7: Stability of the top-weighted questions across different random seeds for 5% and 10% test sets.

	5% Test Set		10% Test Set	
	Overlap (%)	Count	Overlap (%)	Count
Seed 1	91.77	1349 / 1470	87.14	1294 / 1485
Seed 2	91.50	1345 / 1470	85.99	1277 / 1485
Seed 3	90.95	1337 / 1470	86.67	1287 / 1485
Seed 4	91.09	1339 / 1470	86.53	1285 / 1485
Seed 5	91.50	1345 / 1470	85.59	1271 / 1485
Mean $\pm$ Std	$\mathbf{91.36 \pm 0.30}$		$\mathbf{86.38 \pm 0.54}$	

E Baselines

E.1 No Alignment

We aggregate all benchmarks into a single joint feature space. Then, we use the overall accuracy to rank models accordingly.

E.2 Individual

We first split each training benchmark into its constituent tasks. For example, MMLU Pro is subdivided into math, law, psychology, and other subject areas. We then append the benchmark name to each task (e.g., "MMLU Pro – Math") to treat them as individual benchmarks and select the one with the highest Spearman Rank Correlation, which also happened to be the one with the highest Pairwise Ranking Accuracy in all settings.

By treating these tasks as individual benchmarks, we choose the best individual baseline from 52 options rather than the 5 available benchmarks. IFEval is not considered in this selection.

E.3 Random Benchmark

We use the splitted tasks and derive a numerical seed from the resulting string. This seed is used to randomly select a task from the training benchmarks, whose induced model ranking is used directly as the random baseline, while ensuring reproducibility. IFEval is not considered in this selection.

Table 8: Runtime Analysis of Benchmark Alignment in seconds

Number of Models	Learning Rate	Number of Epochs 100	1000	5000	10000
3700	0.0001	14.0	154.0	810.0	941.0
	0.001	17.0	135.0	459.0	933.0
	1	15.0	162.0	646.0	969.0
2000	0.0001	12.0	115.0	731.0	836.0
	0.001	17.0	95.0	537.0	966.0
	1	11.0	118.0	602.0	1112.0
1000	0.0001	7.0	109.0	474.0	1199.0
	0.001	13.0	156.0	581.0	501.0
	1	6.0	108.0	599.0	888.0
500	0.0001	16.0	64.0	553.0	724.0
	0.001	9.0	109.0	443.0	632.0
	1	8.0	104.0	348.0	995.0
100	0.0001	8.0	87.0	456.0	780.0
	0.001	7.0	61.0	309.0	493.0
	1	11.0	99.0	409.0	607.0

E.4 MetaBench

We implement the METABENCH baseline following the general pipeline proposed by Kipnis et al. (2025) with one deliberate deviation: we do not apply the final information-filtering step based on Fisher information. Below, we describe our implementation in detail and motivate this design choice.

E.4.1 Data preprocessing and benchmark construction

We begin from question-level correctness data released as part of the OpenLLM Leaderboard. Following METABENCH, for each benchmark we compute item-level statistics on the training models only, including standard deviation, difficulty (scaled mean accuracy), and part-whole correlation between each item and the total score. Items are excluded if they (i) have near-zero variance, (ii) are excessively easy, or (iii) exhibit low discrimination. This mirrors the preprocessing steps described in Section 2.1 of METABENCH. To avoid information leakage, all item selection is performed strictly on the training portion of each split.

E.4.2 Cross-validated subsampling

After preprocessing, we follow the METABENCH subsampling procedure to select a subset of items. For each split, we repeatedly sample subsets of size k (with k ranging from 500 to 1500 in our experiments), compute subtest scores as mean correctness, and fit a generalized additive model (GAM) on the training models to predict the full benchmark score. Validation RMSE is estimated using five-fold cross-validation, and the subset minimizing this error is retained. This procedure closely follows Section 2.2 of METABENCH, with GAMs used to capture the non-linear relationship between subtest scores and full benchmark scores.

E.4.3 IRT modeling and score prediction

Using the selected item subsets, we fit one-dimensional IRT models (2PL/3PL/4PL depending on configuration) on the training models using the `mirt` package. Latent abilities are estimated via maximum a posteriori (MAP) inference. As in METABENCH, we combine the estimated latent abilities with subtest scores and train a GAM to predict the target benchmark score. In our setting, the target score for each model is defined as its mean correctness across all raw benchmark items. Test performance is evaluated via RMSE between predicted and true scores on held-out models.

E.4.4 Omission of information filtering

METABENCH proposes a final information filtering stage, where items are selected based on Fisher information across quantiles of the estimated ability distribution, with hyperparameters tuned via Bayesian optimization (Section 2.4 in the original paper). We intentionally omit this step in our implementation.

This decision is empirically motivated. Across all splits and benchmarks we evaluated, applying information filtering did not further reduce test RMSE beyond what was already achieved after IRT fitting and GAM-based score prediction. While the IRT step alone typically reduced test RMSE to approximately 1, the subsequent information filtering stage consistently increased test RMSE, despite substantially shrinking the number of retained items (often to fewer than 200).

Since we observed no improvements in the primary metric of interest (test RMSE), we excluded the information filtering step in our experiments. Importantly, this choice aligns with the stated objective of METABENCH: accurate score reconstruction and predictive performance on unseen models. Our results therefore suggest that, at least under our experimental conditions, METABENCH-style information filtering does not provide additional benefits beyond IRT-based ability estimation.

E.5 TinyBenchmarks

We adapt TINYBENCHMARKS (Polo et al., 2024) to our setting by re-running their `anchor` selection routine on each of our (benchmark, train/test split) pairs. All steps below are repeated independently for every pair; no information from held-out test models is used either to choose items or to fit IRT.

E.5.1 Anchor selection

For each split we form the $|\mathcal{M}_{\text{train}}| \times N$ binary response matrix on the training models and fit weighted k-means in item space with $K = 100$ clusters and uniform item weights $1/N$, taking the best of five random restarts. The anchor for each cluster is the item closest to the centroid (Euclidean), and its weight w_i is the fraction of items assigned to that cluster, so $\sum_i w_i = 1$.

E.5.2 IRT fitting

We fit a multidimensional 2PL IRT model on the same training-model responses with `py-irt` using the `multidim_2pl` module of Polo et al. (2024), with $D = 10$ latent dimensions, hierarchical priors, learning rate 0.1, and $2{,}000$ epochs. This yields per-item parameters $(\mathbf{a}_i, b_i)$ and per-train-model abilities $\boldsymbol{\theta}_m$. No items are filtered. For each test model we observe responses only on the 100 anchors and obtain $\hat{\boldsymbol{\theta}}$ by maximum likelihood with item parameters held fixed.

E.5.3 Reported metric

Our score for test model m is the naïve accuracy $\widehat{\text{acc}}_m$ on the 100 selected items, and the per-split ranking is obtained by sorting test models in descending order of $\widehat{\text{acc}}_m$. For completeness we also compute the p-IRT and gp-IRT estimators of Polo et al. (2024) and log them as auxiliary outputs, but they do not enter the reported results.

F Model Scale Experiments

We obtain model parameter counts by querying the Hugging Face Hub API using the model identifiers provided by OpenLLMLeaderboard. When parameter information is unavailable, we approximate the number of parameters using string matching. For example, if "7B" appears in a model name, we assume the model has 7 billion parameters exactly.

Using this procedure, we recover parameter counts for only 3,994 of the 4,364 models for which complete benchmark data from OpenLLMLeaderboard was available; among these, 434 models require string matching.

From this set, we construct the following size-based source-target groups:

- $>$ 70B: 142 target models and 3852 source models

Table 9: Comparison of learning-to-rank objectives.

Method	Description
RANKNET	First Learning-to-Rank method using gradient descent. Assumes every pair of items has equal importance during training.
LAMBDARANK	Reweights all pairwise orderings according to their impact on a target metric that measures the gains and losses of being assigned a position in the ranking.
NDCG-LOSS1	Reweights all pairings according to a loss function directly derived from the NDCG, rather than incorporating the metric value explicitly during optimization.
NDCG-LOSS2	Derived as an alternative implementation of NDCG-LOSS1 with more constrained loss values when the rank of one of the elements in the pair is too large.
NDCG-LOSS2++	Hybrid method that linearly combines the LAMBDARANK optimization objective with NDCG-LOSS2.

Table 10: Learning-to-rank Ablations

RM	Method	Acc_{pair}	ρ
ArmoRM (Helpful)	RANKNET	0.765 ± 0.005	0.710 ± 0.056
	LAMBDARANK	0.704 ± 0.004	0.564 ± 0.078
	NDCG-LOSS1	0.762 ± 0.004	0.703 ± 0.057
	NDCG-LOSS2	0.755 ± 0.004	0.697 ± 0.058
	NDCG-LOSS2++	0.727 ± 0.005	0.621 ± 0.070
	NO LEARNING-TO-RANK	0.605 ± 0.005	0.331 ± 0.105
DPA	RANKNET	0.763 ± 0.005	0.713 ± 0.055
	LAMBDARANK	0.711 ± 0.004	0.597 ± 0.073
	NDCG-LOSS1	0.753 ± 0.004	0.696 ± 0.058
	NDCG-LOSS2	0.770 ± 0.004	0.727 ± 0.053
	NDCG-LOSS2++	0.723 ± 0.005	0.630 ± 0.069
	NO LEARNING-TO-RANK	0.609 ± 0.005	0.318 ± 0.106

- $>$ 30B: 255 target models and 3739 source models
- $>$ 13B: 845 target models and 3149 source models

G Ablation Studies

G.1 Learning-to-rank Algorithms

We study the effect of learning-to-rank by comparing the performance of the five algorithms described in Table 9 against the ranking performance based on directly aggregating scores from all benchmarks, denoted by NO LEARNING-TO-RANK. All algorithms are drawn from Wang et al. (2018), where they are systematically compared across multiple ranking settings, making them suitable for constructing alternative versions of BENCHALIGN. We refer readers to Wang et al. (2018) for implementation details. We perform our evaluation using the ArmoRM (Helpful) and DPA preferences on the 30B+ target from Section 4.2.

Table 10 reports the ranking performance of the five learning-to-rank methods and the baseline without learning-to-rank. We find that NO LEARNING-TO-RANK yields a ranking that is positively correlated with our target preferences based on helpfulness and honesty. This suggests that there may be a potential overlap between models' capabilities and their perceived preferences on these benchmark tasks. The incorporation of learning-to-rank enables BENCHALIGN to learn benchmark weightings that better align with the target preference ranking, with all methods achieving Acc_{pair} above 0.7. They also outperform around two times higher than NO LEARNING-TO-RANK baseline

Table 11: Helpsteer and UltraFeedback results under arbitrary model sets

Target	Method	Helpsteer				UltraFeedback			
		Acc_{pair}		ρ		Acc_{pair}		ρ	
		ArmoRM	*GPT2*	*ArmoRM*	*GPT2*	*ArmoRM*	*DPA*	*ArmoRM*	*DPA*
2%	NO ALIGNMENT	0.720 ± 0.014	0.633 ± 0.015	0.614 ± 0.151	0.383 ± 0.196	0.736 ± 0.014	0.724 ± 0.014	0.652 ± 0.140	0.627 ± 0.147
	RANDOM	0.675 ± 0.015	0.587 ± 0.016	0.385 ± 0.181	0.252 ± 0.198	0.739 ± 0.014	0.733 ± 0.014	0.666 ± 0.120	0.655 ± 0.123
	INDIVIDUAL	0.733 ± 0.014	0.675 ± 0.015	0.637 ± 0.145	0.464 ± 0.183	0.750 ± 0.014	0.733 ± 0.014	0.680 ± 0.136	0.648 ± 0.142
	METABENCH	0.727 ± 0.010	0.632 ± 0.011	0.624 ± 0.131	0.386 ± 0.181	0.746 ± 0.010	0.748 ± 0.010	0.698 ± 0.111	0.689 ± 0.113
	TINYBENCHMARKS	0.713 ± 0.014	0.626 ± 0.016	0.598 ± 0.137	0.375 ± 0.182	0.729 ± 0.014	0.716 ± 0.014	0.638 ± 0.127	0.614 ± 0.134
	BENCHALIGN	0.866 ± 0.011	0.807 ± 0.013	$\mathbf{0.895 \pm 0.035}$	$\mathbf{0.800 \pm 0.078}$	$\mathbf{0.891 \pm 0.010}$	$\mathbf{0.891 \pm 0.010}$	$\mathbf{0.933 \pm 0.028}$	$\mathbf{0.929 \pm 0.030}$
5%	NO ALIGNMENT	0.708 ± 0.006	0.605 ± 0.006	0.580 ± 0.095	0.314 ± 0.125	0.727 ± 0.006	0.722 ± 0.006	0.630 ± 0.088	0.616 ± 0.090
	RANDOM	0.670 ± 0.006	0.589 ± 0.006	0.384 ± 0.114	0.257 ± 0.124	0.715 ± 0.006	0.706 ± 0.006	0.614 ± 0.084	0.594 ± 0.087
	INDIVIDUAL	0.722 ± 0.006	0.639 ± 0.006	0.611 ± 0.091	0.379 ± 0.120	0.747 ± 0.006	0.737 ± 0.006	0.669 ± 0.081	0.648 ± 0.084
	METABENCH	0.708 ± 0.004	0.604 ± 0.004	0.581 ± 0.089	0.312 ± 0.120	0.718 ± 0.004	0.715 ± 0.004	0.627 ± 0.081	0.611 ± 0.084
	TINYBENCHMARKS	0.703 ± 0.006	0.602 ± 0.006	0.570 ± 0.090	0.312 ± 0.120	0.722 ± 0.006	0.715 ± 0.006	0.620 ± 0.082	0.605 ± 0.085
	BENCHALIGN	0.849 ± 0.005	$\mathbf{0.792 \pm 0.005}$	0.866 ± 0.034	0.762 ± 0.056	$\mathbf{0.870 \pm 0.004}$	$\mathbf{0.867 \pm 0.004}$	$\mathbf{0.905 \pm 0.024}$	$\mathbf{0.899 \pm 0.026}$
10%	NO ALIGNMENT	0.678 ± 0.003	0.590 ± 0.003	0.502 ± 0.074	0.264 ± 0.090	0.697 ± 0.003	0.695 ± 0.003	0.558 ± 0.068	0.551 ± 0.069
	RANDOM	0.652 ± 0.003	0.580 ± 0.003	0.345 ± 0.083	0.223 ± 0.089	0.704 ± 0.003	0.696 ± 0.003	0.588 ± 0.062	0.571 ± 0.064
	INDIVIDUAL	0.701 ± 0.003	0.632 ± 0.003	0.545 ± 0.070	0.356 ± 0.085	0.713 ± 0.003	0.724 ± 0.003	0.589 ± 0.065	0.598 ± 0.064
	METABENCH	0.674 ± 0.002	0.585 ± 0.002	0.493 ± 0.071	0.252 ± 0.088	0.702 ± 0.002	0.698 ± 0.002	0.580 ± 0.063	0.566 ± 0.064
	TINYBENCHMARKS	0.675 ± 0.003	0.589 ± 0.003	0.494 ± 0.071	0.264 ± 0.087	0.693 ± 0.003	0.690 ± 0.003	0.550 ± 0.066	0.539 ± 0.067
	BENCHALIGN	$\mathbf{0.855 \pm 0.002}$	$\mathbf{0.814 \pm 0.002}$	$\mathbf{0.876 \pm 0.022}$	$\mathbf{0.804 \pm 0.033}$	0.863 ± 0.002	0.862 ± 0.002	0.894 ± 0.019	0.891 ± 0.019

on the rank correlation ρ. These results demonstrate that learning-to-rank is effective for aligning benchmarks with target preferences.

G.2 Generalization under Arbitrary Model Sets

In Sections 4.2, we are curating the models that are part of our train-test splits according to the model size. As conditioning on a model's characteristics could impact our previous findings, we study the effect conditioning has on benchmark alignment by evaluating our baselines using three source-target splits formed at random, holding out 2%, 5%, 10% of all models as target.

Table 11 reports the ranking performance on all target sets. We observe that BENCHALIGN produces better aligned benchmarks under random splits than when conditioning on model scale, achieving an increase of around 0.1 points in Acc_{pair} and ρ in most cases. This comparison suggests that alignment benefits from the greater variability introduced by random sampling into the model pool. The metrics achieved by the remaining baselines provide further evidence of the gains obtained from an unconditioned sampling, as all of them also output rankings of unseen models with higher Acc_{pair} and ρ.

However, the results in $GPT2$ provide further insights on generalization. $GPT2$ is the worst performing setting from all experiments and also one of the settings where the correlation obtained by BENCHALIGN increases the most. BENCHALIGN correlations in Section 4.2 remain between 0.3 and 0.5, while TINYBENCHMARKS and METABENCH struggle to output significant correlations. In arbitrary model sets, these correlations increase to values between 0.76 and 0.8 for BENCHALIGN, and between 0.2 and 0.3 for both TINYBENCHMARKS and METABENCH. As the results of TINYBENCHMARKS and METABENCH are the equivalent of using the overall accuracy in all benchmarks as a proxy for preferences, these results suggest that benchmark data with small or no correlation with the target preferences might still provide useful information for alignment as long as there is enough variability in the data available during training.

G.3 Preference Heterogeneity

We evaluate the robustness of our approach to preference heterogeneity by doing an additional set of experiments considering preferences regarding whether a human will judge the responses of a model better than other. This preferences were obtained by training reward models on WebGPT[16], Summarize from Feedback[17] and Synthethic Instruct GPT[18] and the base architectures correspond to He et al. (2021). All reward models below were trained differently on these 3 datasets, making them suitable to evaluate heterogeneity. Furthermore, we evaluate the responses of all models in Math Precalculus Hard rather than IFEval as finding positive results in Math would show that the results for heterogeneity are not exclusive to the benchmark we used in previous settings. We perform our

[16] `https://huggingface.co/datasets/openai/webgpt_comparisons`

[17] `https://huggingface.co/datasets/openai/summarize_from_feedback`

[18] `Dahoas/synthetic-instruct-gptj-pairwise`

evaluation on 3 randomly splitted target sets: 2%, 5% and 10%. Specifically, we consider four reward models[19]:

- RM_1^{judge}: Deberta-v3-base
- RM_2^{judge}: Deberta-v3-large
- RM_3^{judge}: Deberta-v3-large-v2
- RM_4^{judge}: Electra-Large-Discriminator

[19]The implementations used are from OpenAssistant and can be found here: `https://huggingface.co/OpenAssistant`

Table 12: Results under Math Precalculus Hard as evaluation dataset and heterogenous preference sources

Target	Method	Acc_{pair}				ρ			
		RM_1^{Judge}	RM_2^{Judge}	RM_3^{Judge}	RM_4^{Judge}	RM_1^{Judge}	RM_2^{Judge}	RM_3^{Judge}	RM_4^{Judge}
2%	NO ALIGNMENT	0.578 ± 0.001	0.743 ± 0.007	0.638 ± 0.015	0.684 ± 0.015	0.272 ± 0.207	0.700 ± 0.122	0.390 ± 0.191	0.511 ± 0.174
	RANDOM	0.606 ± 0.016	0.760 ± 0.014	0.650 ± 0.020	0.681 ± 0.015	0.299 ± 0.209	0.717 ± 0.121	0.431 ± 0.199	0.506 ± 0.175
	INDIVIDUAL	0.649 ± 0.015	0.773 ± 0.031	0.709 ± 0.015	0.729 ± 0.014	0.395 ± 0.194	0.742 ± 0.112	0.548 ± 0.167	0.598 ± 0.155
	METABENCH	0.581 ± 0.011	0.748 ± 0.010	0.631 ± 0.011	0.678 ± 0.011	0.244 ± 0.199	0.701 ± 0.110	0.383 ± 0.181	0.503 ± 0.159
	TINYBENCHMARKS	0.604 ± 0.016	0.745 ± 0.014	0.647 ± 0.015	0.679 ± 0.015	0.300 ± 0.193	0.695 ± 0.111	0.418 ± 0.175	0.507 ± 0.158
	BENCHALIGN	$\mathbf{0.768 \pm 0.014}$	$\mathbf{0.842 \pm 0.012}$	$\mathbf{0.820 \pm 0.012}$	$\mathbf{0.773 \pm 0.013}$	$\mathbf{0.698 \pm 0.127}$	$\mathbf{0.855 \pm 0.065}$	$\mathbf{0.824 \pm 0.082}$	$\mathbf{0.693 \pm 0.128}$
5%	NO ALIGNMENT	0.559 ± 0.001	0.713 ± 0.006	0.610 ± 0.006	0.616 ± 0.006	0.193 ± 0.131	0.606 ± 0.091	0.327 ± 0.124	0.336 ± 0.123
	RANDOM	0.574 ± 0.006	0.713 ± 0.006	0.623 ± 0.006	0.609 ± 0.006	0.209 ± 0.131	0.591 ± 0.091	0.358 ± 0.122	0.309 ± 0.125
	INDIVIDUAL	0.607 ± 0.006	0.724 ± 0.007	0.670 ± 0.006	0.671 ± 0.006	0.277 ± 0.127	0.643 ± 0.085	0.453 ± 0.112	0.460 ± 0.112
	METABENCH	0.552 ± 0.004	0.713 ± 0.004	0.601 ± 0.004	0.612 ± 0.004	0.159 ± 0.130	0.603 ± 0.085	0.305 ± 0.121	0.325 ± 0.119
	TINYBENCHMARKS	0.569 ± 0.006	0.701 ± 0.006	0.623 ± 0.006	0.608 ± 0.006	0.209 ± 0.127	0.577 ± 0.089	0.363 ± 0.116	0.312 ± 0.120
	BENCHALIGN	$\mathbf{0.770 \pm 0.005}$	$\mathbf{0.850 \pm 0.005}$	$\mathbf{0.811 \pm 0.005}$	$\mathbf{0.803 \pm 0.005}$	$\mathbf{0.705 \pm 0.074}$	$\mathbf{0.873 \pm 0.036}$	$\mathbf{0.806 \pm 0.052}$	$\mathbf{0.770 \pm 0.060}$
10%	NO ALIGNMENT	0.563 ± 0.003	0.713 ± 0.003	0.605 ± 0.003	0.626 ± 0.003	0.184 ± 0.094	0.601 ± 0.000	0.314 ± 0.087	0.363 ± 0.084
	RANDOM	0.565 ± 0.003	0.706 ± 0.003	0.612 ± 0.003	0.619 ± 0.003	0.181 ± 0.092	0.576 ± 0.066	0.323 ± 0.087	0.337 ± 0.086
	INDIVIDUAL	0.622 ± 0.003	0.728 ± 0.003	0.682 ± 0.003	0.664 ± 0.003	0.327 ± 0.087	0.622 ± 0.061	0.492 ± 0.075	0.448 ± 0.078
	METABENCH	0.557 ± 0.002	0.715 ± 0.002	0.598 ± 0.002	0.625 ± 0.002	0.168 ± 0.091	0.606 ± 0.060	0.296 ± 0.086	0.363 ± 0.082
	TINYBENCHMARKS	0.566 ± 0.003	0.701 ± 0.003	0.613 ± 0.003	0.619 ± 0.003	0.195 ± 0.090	0.576 ± 0.063	0.335 ± 0.083	0.348 ± 0.083
	BENCHALIGN	$\mathbf{0.782 \pm 0.003}$	$\mathbf{0.850 \pm 0.002}$	$\mathbf{0.814 \pm 0.002}$	$\mathbf{0.808 \pm 0.003}$	$\mathbf{0.746 \pm 0.045}$	$\mathbf{0.875 \pm 0.024}$	$\mathbf{0.814 \pm 0.034}$	$\mathbf{0.796 \pm 0.037}$

Target Models	**Method**	Acc_{pair}				ρ			
		RM_1^{Judge}	RM_2^{Judge}	RM_3^{Judge}	RM_4^{Judge}	RM_1^{Judge}	RM_2^{Judge}	RM_3^{Judge}	RM_4^{Judge}
70B+	NO ALIGNMENT	0.482 ± 0.010	0.607 ± 0.010	0.509 ± 0.010	0.541 ± 0.010	-0.051 ± 0.166	0.318 ± 0.141	0.030 ± 0.164	0.163 ± 0.156
	RANDOM	0.555 ± 0.010	0.583 ± 0.010	$\mathbf{0.632 \pm 0.009}$	0.541 ± 0.010	0.123 ± 0.159	0.236 ± 0.150	0.322 ± 0.140	0.123 ± 0.159
	INDIVIDUAL	0.583 ± 0.010	0.668 ± 0.009	$\mathbf{0.632 \pm 0.009}$	0.647 ± 0.009	0.198 ± 0.153	0.465 ± 0.120	0.322 ± 0.140	0.372 ± 0.134
	METABENCH	0.481 ± 0.007	0.611 ± 0.007	0.498 ± 0.007	0.546 ± 0.007	-0.059 ± 0.164	0.331 ± 0.147	0.000 ± 0.165	0.177 ± 0.160
	TINYBENCHMARKS	0.457 ± 0.007	0.585 ± 0.007	0.511 ± 0.001	0.542 ± 0.007	-0.113 ± 0.163	0.267 ± 0.153	0.032 ± 0.067	0.142 ± 0.162
	BENCHALIGN	$\mathbf{0.620 \pm 0.010}$	$\mathbf{0.735 \pm 0.009}$	0.617 ± 0.010	$\mathbf{0.712 \pm 0.009}$	$\mathbf{0.340 \pm 0.138}$	$\mathbf{0.647 \pm 0.087}$	$\mathbf{0.338 \pm 0.138}$	$\mathbf{0.603 \pm 0.095}$
30B+	NO ALIGNMENT	0.433 ± 0.005	0.619 ± 0.005	0.482 ± 0.005	0.569 ± 0.005	-0.186 ± 0.121	0.351 ± 0.103	-0.048 ± 0.123	0.229 ± 0.113
	RANDOM	0.467 ± 0.005	0.605 ± 0.005	0.630 ± 0.005	0.544 ± 0.005	-0.112 ± 0.123	0.302 ± 0.108	0.335 ± 0.105	0.133 ± 0.119
	INDIVIDUAL	0.549 ± 0.005	0.666 ± 0.005	0.630 ± 0.005	0.622 ± 0.005	0.114 ± 0.120	0.466 ± 0.091	0.335 ± 0.105	0.340 ± 0.104
	METABENCH	0.406 ± 0.004	0.601 ± 0.004	0.449 ± 0.004	0.554 ± 0.004	-0.268 ± 0.114	0.294 ± 0.112	-0.141 ± 0.120	0.177 ± 0.119
	TINYBENCHMARKS	0.432 ± 0.004	0.606 ± 0.004	0.491 ± 0.004	0.561 ± 0.004	-0.179 ± 0.119	0.315 ± 0.111	-0.028 ± 0.123	0.194 ± 0.118
	BENCHALIGN	$\mathbf{0.621 \pm 0.005}$	$\mathbf{0.737 \pm 0.005}$	$\mathbf{0.677 \pm 0.005}$	$\mathbf{0.721 \pm 0.005}$	$\mathbf{0.348 \pm 0.104}$	$\mathbf{0.645 \pm 0.067}$	$\mathbf{0.500 \pm 0.087}$	$\mathbf{0.606 \pm 0.072}$
13B+	NO ALIGNMENT	0.426 ± 0.002	0.603 ± 0.002	0.495 ± 0.002	0.529 ± 0.002	-0.215 ± 0.065	0.298 ± 0.060	-0.016 ± 0.068	0.081 ± 0.067
	RANDOM	0.454 ± 0.002	0.599 ± 0.002	0.581 ± 0.002	0.509 ± 0.002	-0.156 ± 0.067	0.277 ± 0.061	0.224 ± 0.063	0.030 ± 0.067
	INDIVIDUAL	0.587 ± 0.002	0.660 ± 0.002	$\mathbf{0.652 \pm 0.002}$	0.612 ± 0.002	0.223 ± 0.063	0.451 ± 0.052	0.408 ± 0.055	0.319 ± 0.059
	METABENCH	0.432 ± 0.001	0.621 ± 0.001	0.492 ± 0.001	0.540 ± 0.001	-0.194 ± 0.065	0.343 ± 0.060	-0.025 ± 0.067	0.114 ± 0.067
	TINYBENCHMARKS	0.440 ± 0.001	0.579 ± 0.001	0.508 ± 0.001	0.520 ± 0.001	-0.175 ± 0.065	0.232 ± 0.064	0.025 ± 0.067	0.060 ± 0.067
	BENCHALIGN	$\mathbf{0.591 \pm 0.002}$	$\mathbf{0.709 \pm 0.001}$	0.646 ± 0.002	$\mathbf{0.690 \pm 0.002}$	$\mathbf{0.261 \pm 0.062}$	$\mathbf{0.571 \pm 0.044}$	$\mathbf{0.417 \pm 0.054}$	$\mathbf{0.520 \pm 0.048}$

Table 13: Alignment with heterogeneous preferences under model scale splits

We observe in Tables 13 and 12 that our simple alignment approach outperforms all cases, including the INDIVIDUAL baseline. In the 10% target, this difference is even stronger, as the difference between accuracies of these two baselines is of $\sim 18\%$ in RM_1^{Judge}, while the difference in the correlations is much larger, reaching approximately 42%. The NO ALIGNMENT baseline shows further limitations of benchmarks under these conditions as the correlations are low and inconsistent across preferences. Overall, our results suggest that the alignment can be performed across heterogeneous sets of preferences, even when the benchmarks used don't have a stable correlation with the target preferences.

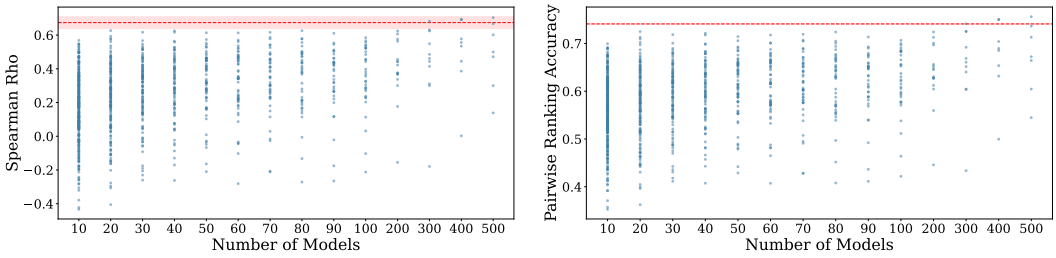
H Minimizing Data Requirements across Multiple Preferences and Targets

H.1 Number of Models

We consider the best smallest subsets that enable alignment from the ones with a correlation or accuracy whose confidence bands overlap those of using all training models. Table 14 shows a summary of these subsets for all experiments corresponding to Section 4.2. For all cases but GPT2, the windows are slightly above the average and median number of parameters, but none of the cases are close to the maximum number of parameters from their corresponding training set (13B, 30B or 70B). Surprisingly, these results indicate that the model size is relevant, but not crucial for determining the best subset of models. In the following subsections, we show the figures corresponding to each set of experiments.

Table 14: Number of Parameters for Best Subset of Models

RM	Target	Number of Models	Average Size of Best Subset	Average Size - All Subsets	Median Size - All Subsets
ArmoRM (Helpful)	13B+	20	8.030	6.361	7.616
ArmoRM (Helpful)	30B+	10	14.770	7.918	8.000
ArmoRM (Helpful)	70B+	10	14.770	8.320	8.008
GPT2	13B+	400	7.823	6.503	7.626
GPT2	30B+	10	0.385	7.918	8.000
GPT2	70B+	10	1.257	8.320	8.008
ArmoRM (Honest)	13B+	20	8.030	6.361	7.616
ArmoRM (Honest)	30B+	10	14.766	7.918	8.000
ArmoRM (Honest)	70B+	10	12.100	8.320	8.008
DPA	13B+	20	8.030	6.361	7.616
DPA	30B+	10	14.766	7.918	8.000
DPA	70B+	10	7.616	8.320	8.008



(a) Spearman Rho (ρ)

(b) Pairwise Ranking Accuracy (Acc_{pair})

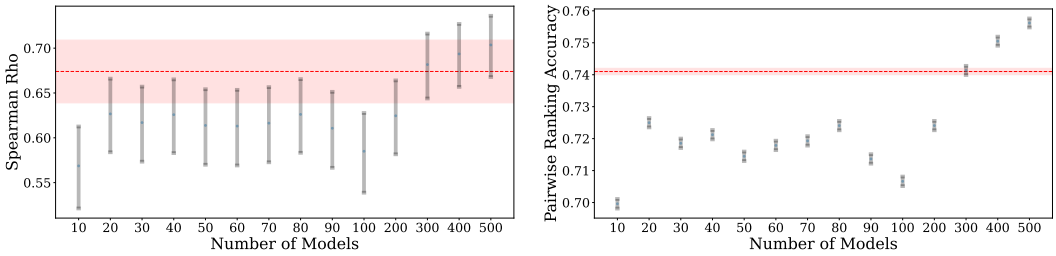
Figure 14: ArmoRM (Helpful) - Target 13B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.

H.1.1 ArmoRM (Helpful)

Target: 13B+. Figures 15 and 15 represent the results of all subset of models and the best subset by number of models, respectively.

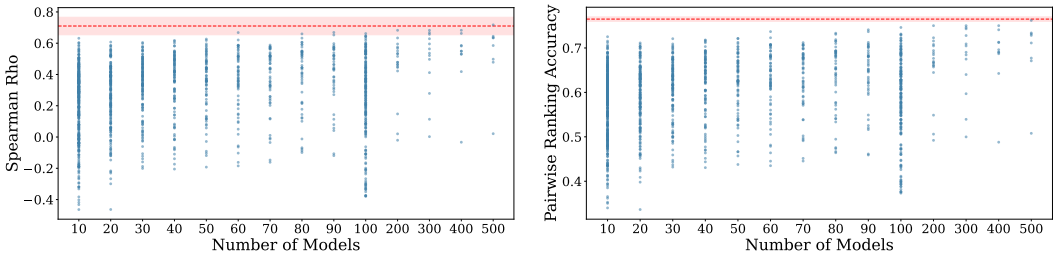
Target: 30B+. Figures 17 and 17 represent the results of all subset of models and the best subset by number of models, respectively.

Target: 70B+. Figures 19 and 19 represent the results of all subset of models and the best subset by number of models, respectively.



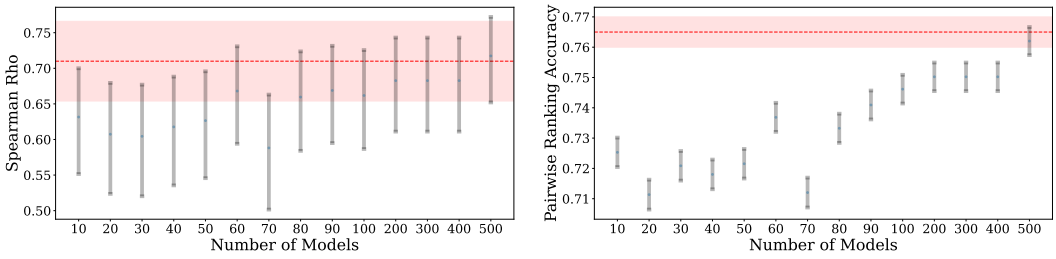
(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 15: ArmoRM (Helpful) - Target 13B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



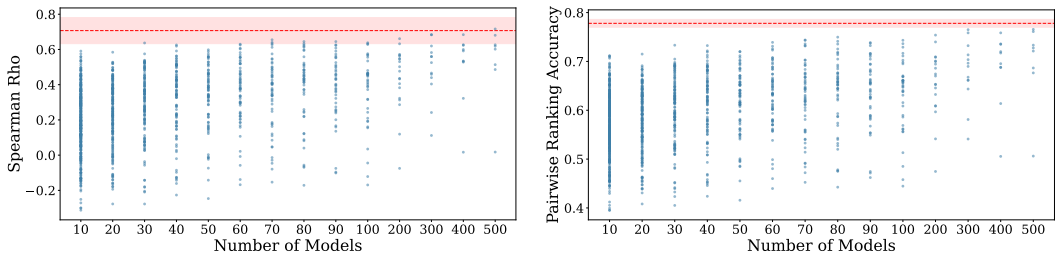
(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 16: ArmoRM (Helpful) - Target 30B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



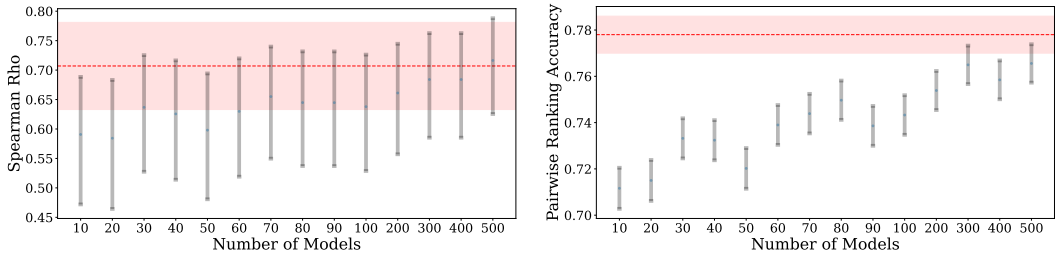
(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 17: ArmoRM (Helpful) - Target 30B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



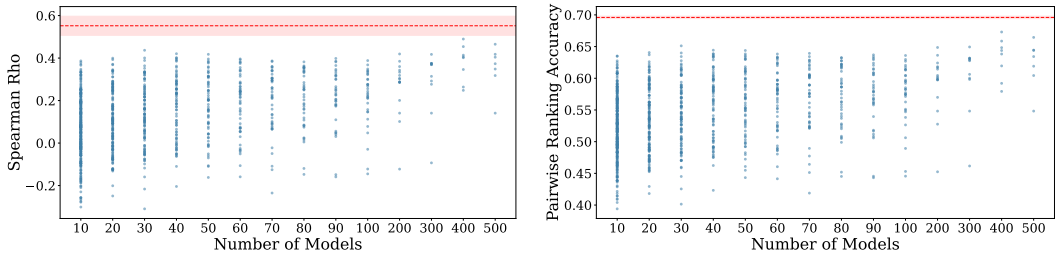
(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 18: ArmoRM (Helpful) - Target 70B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



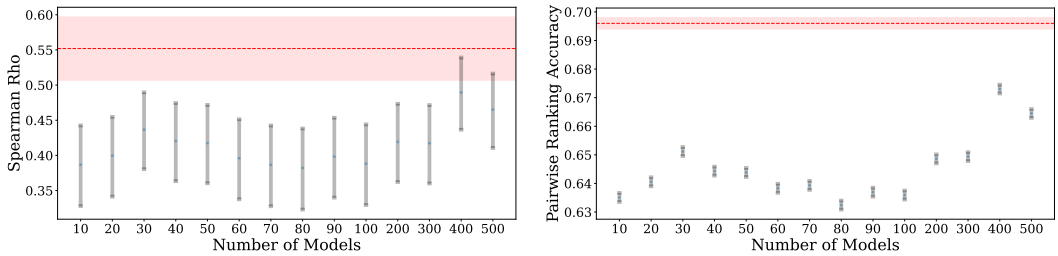
(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 19: ArmoRM (Helpful) - Target 70B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 20: GPT2 - Target 13B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

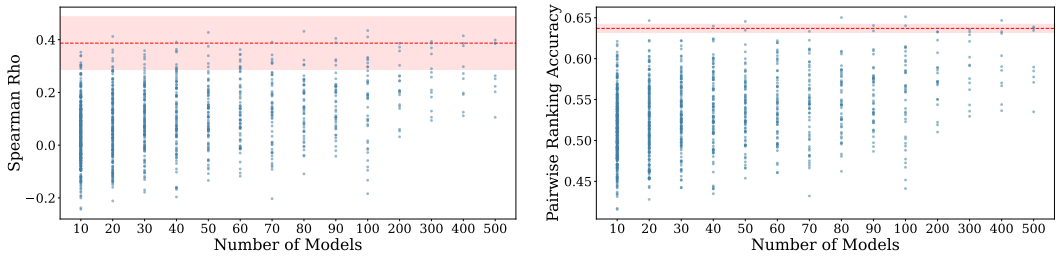
Figure 21: GPT2 - Target 13B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.

H.1.2 GPT2

Target: 13B+.. Figures 21 and 21 represent the results of all subset of models and the best subset by number of models, respectively.

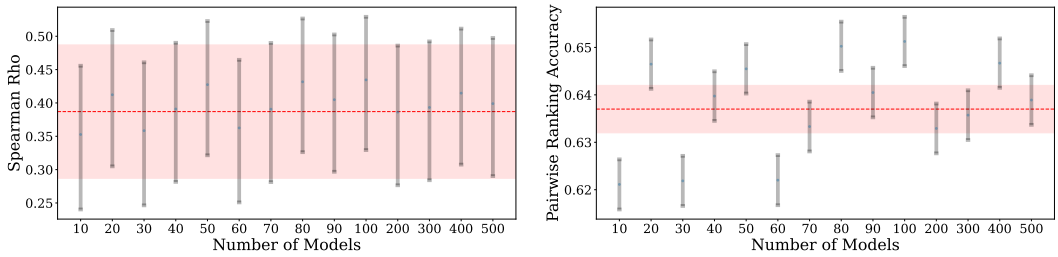
Target: 30B+.. Figures 23 and 23 represent the results of all subset of models and the best subset by number of models, respectively.

Target: 70B+.. Figures 25 and 25 represent the results of all subset of models and the best subset by number of models, respectively.



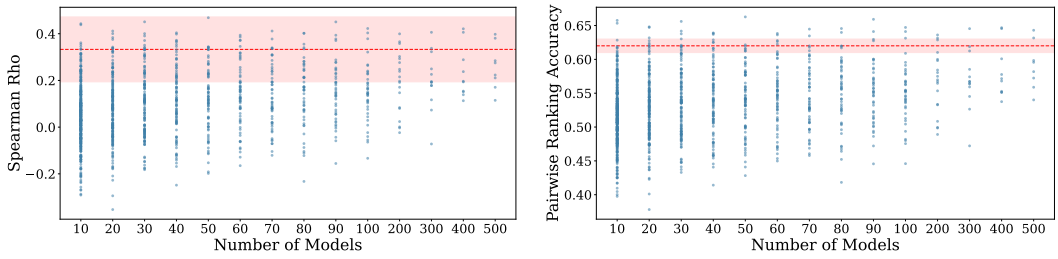
(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 22: GPT2 - Target 30B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



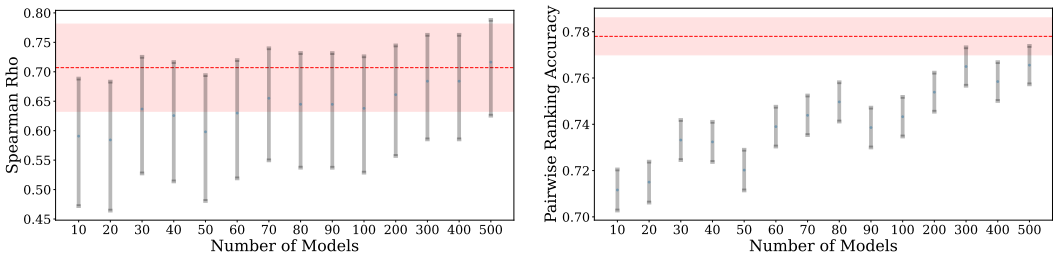
(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 23: GPT2 - Target 30B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

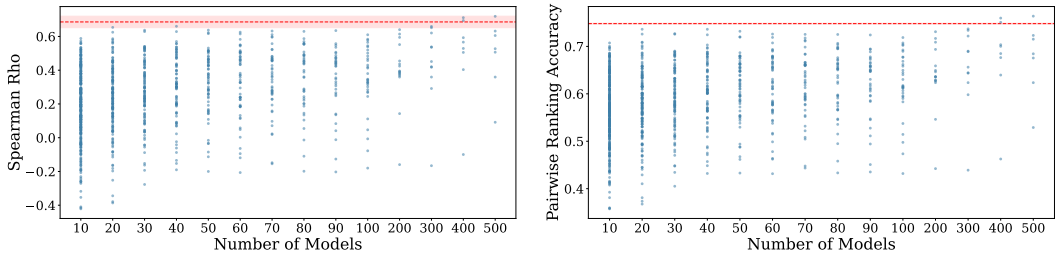
Figure 24: GPT2 - Target 70B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ)

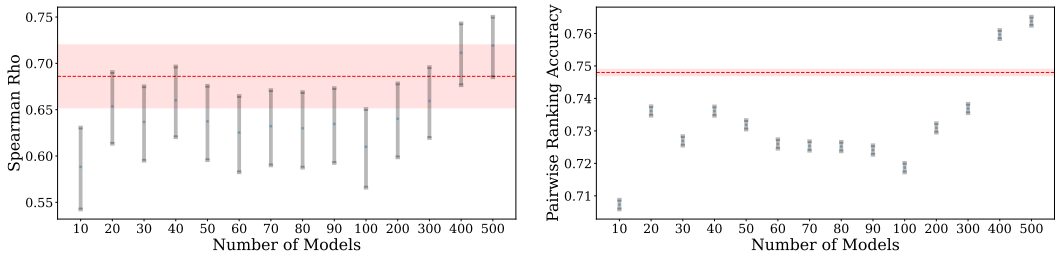
(b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 25: GPT2 - Target 70B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 26: ArmoRM (Honest) - Target 13B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

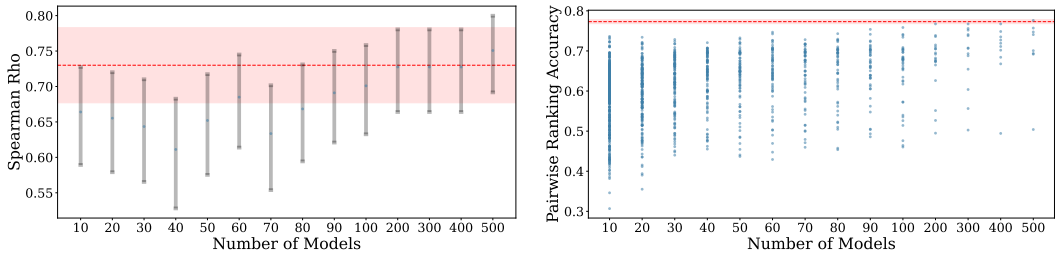
Figure 27: ArmoRM (Honest) - Target 13B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.

H.1.3 ArmoRM (Honest)

Target: 13B+. Figures 27 and 27 represent the results of all subset of models and the best subset by number of models, respectively.

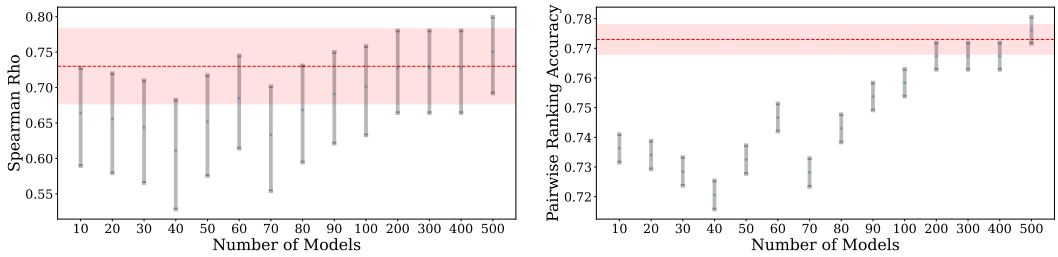
Target: 30B+. Figures 29 and 29 represent the results of all subset of models and the best subset by number of models, respectively.

Target: 70B+. Figures 31 and 31 represent the results of all subset of models and the best subset by number of models, respectively.



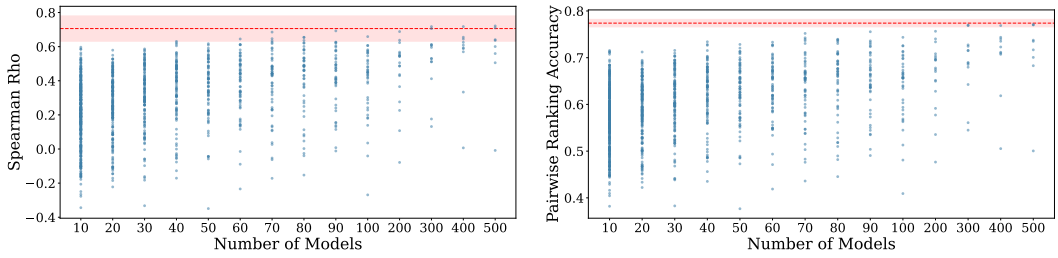
(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 28: ArmoRM (Honest) - Target 30B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



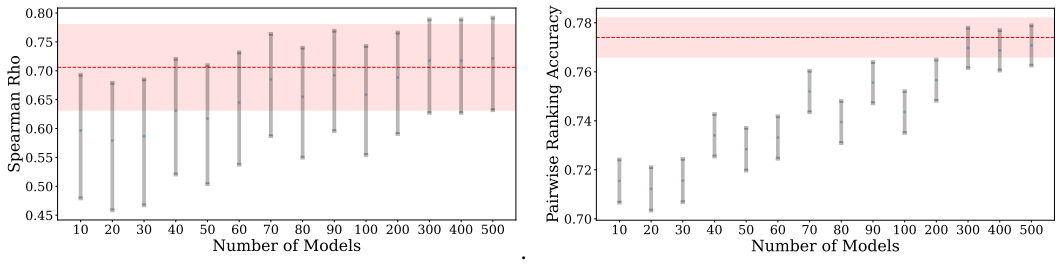
(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 29: ArmoRM (Honest) - Target 30B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

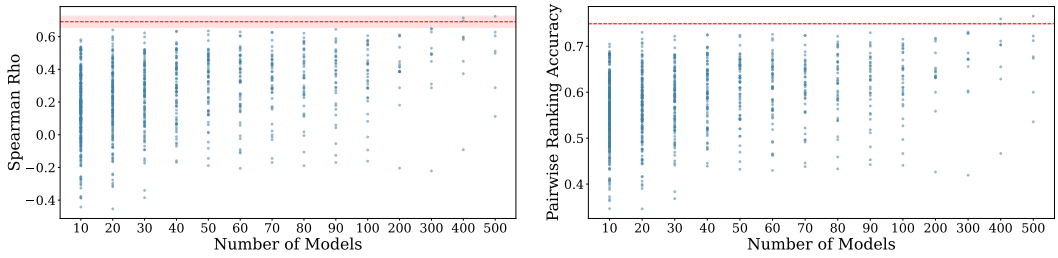
Figure 30: ArmoRM (Honest) - Target 70B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ)

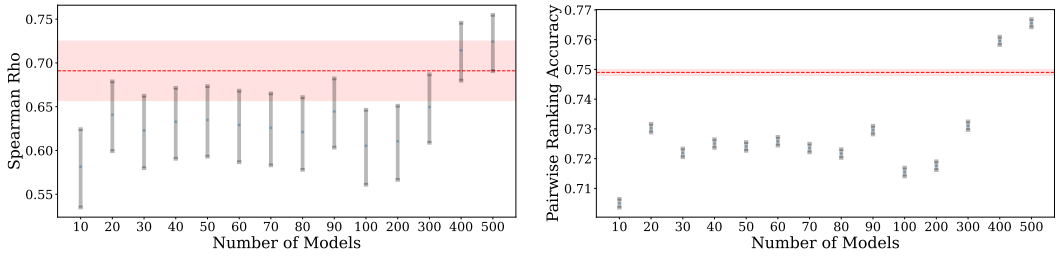
(b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 31: ArmoRM (Honest) - Target 70B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 32: DPA - Target 13B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 33: DPA - Target 13B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.

H.1.4 DPA

Target: 13B+. Figures 33 and 33 represent the results of all subset of models and the best subset by number of models, respectively.

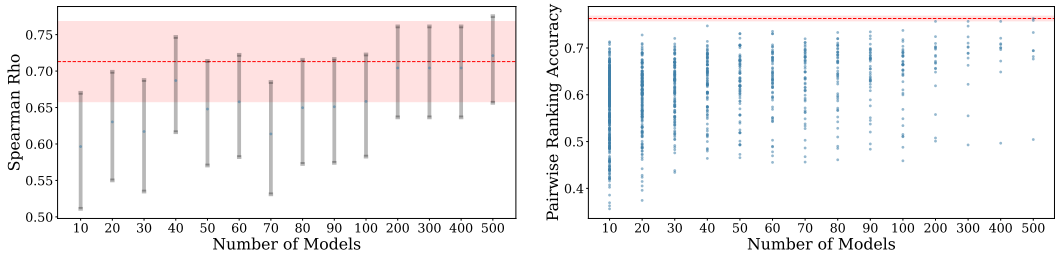
Target: 30B+. Figures 35 and 35 represent the results of all subset of models and the best subset by number of models, respectively.

Target: 70B+. Figures 31 and 31 represent the results of all subset of models and the best subset by number of models, respectively.

H.2 Number of Questions

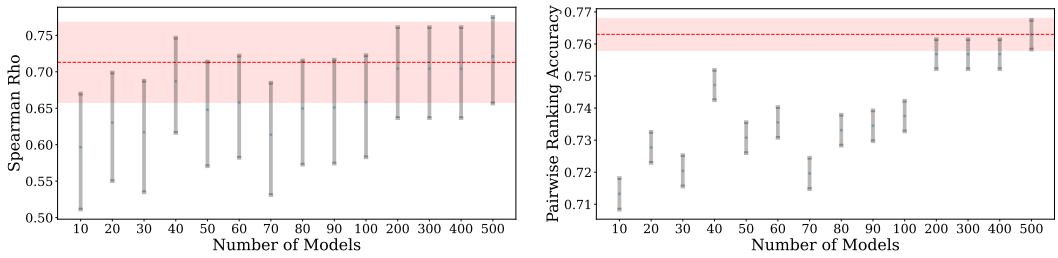
We extend the analysis from Section 4.4 by evaluating BENCHALIGN in combination with TINYBENCHMARKS and METABENCH on every preference and target considered in Section 4.2 and Appendix G.3. Both approaches probe how few question items are sufficient to retain alignment, but they do so at very different scales: TINYBENCHMARKS keeps a fixed set of 100 items per benchmark (around 5000 items in total across all OpenLLMLeaderboard benchmarks), while METABENCH selects only 1500 items in total across all of the benchmarks.

Model scale splits. Table 15 reports for all model-scale splits. Distillation-plus-alignment combinations remain far above the NO ALIGNMENT baseline at every target. Simultaneously, the gap between using all questions for alignment and the two distillation variants narrows as the target grows: at 70B+, BENCHALIGN achieves the best results in all columns, but at 13B+ TINYBENCHMARKS+BENCHALIGN matches or outperforms all baselines in most cases using around 5000 items. These results suggest that the small fraction of questions obtained by benchmark distillation contain sufficient preference signals to enable alignment.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 34: DPA - Target 30B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.

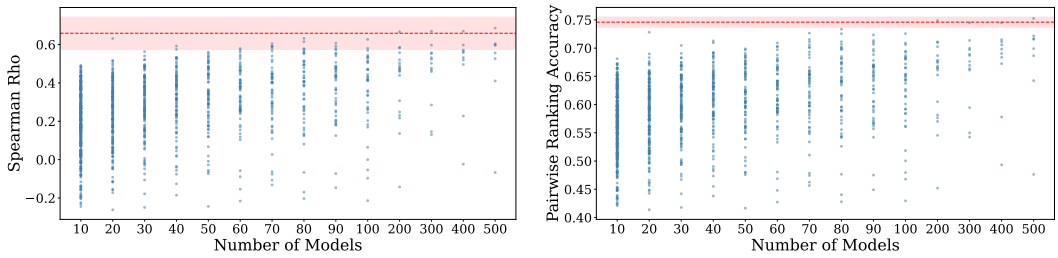


(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 35: DPA - Target 30B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.

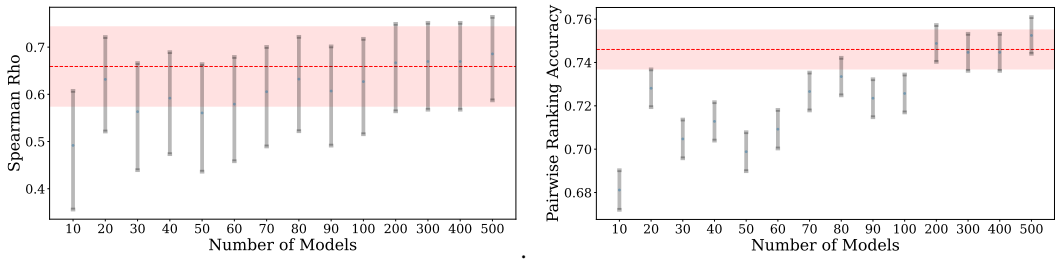
Heterogeneous preferences. Table 16 shows that benchmark alignment plus distillation under heterogeneous preferences shows similar patterns to the ones observed in Table 15. Both distillation-plus-alignment variants recover most of the gains provided by full BENCHALIGN over NO ALIGNMENT. METABENCH+BENCHALIGN improves Acc_{pair} by roughly 14–19 points and raises ρ to the 0.6–0.8 range across all targets, despite using only 1,500 items. TINYBENCHMARKS+BENCHALIGN is still the best alternative as it consistently achieves alignment with a reduced number of questions. Overall, the carefully selected subsets recover nearly all of the alignment gains achieved using the full item pool under heterogenous preferences.

Arbitrary model splits. To show the robustness of our findings further, we repeat our distillation experiments using the same targets as 4.2 but under arbitrary splits. Table 17 shows the results across all target models and preferences, TINYBENCHMARKS+BENCHALIGN matches full BENCHALIGN within confidence intervals on both Acc_{pair} and ρ, and occasionally slightly surpasses it. METABENCH+BENCHALIGN, despite using only around 1500 items, remains within roughly 3–5 points of BENCHALIGN's accuracy while substantially outperforming the NO ALIGNMENT baseline on both metrics. Together with the model-scale and heterogeneous-preference experiments, these results indicate that benchmark alignment does not require access to the a large number of test items to be performed as long as we know in advance which of them contain the largest preference signals.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 36: DPA - Target 70B+. Aligned benchmarks created with different numbers of models. In (a), each point shows the Spearman Rho for one subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.



(a) Spearman Rho (ρ) (b) Pairwise Ranking Accuracy (Acc_{pair})

Figure 37: DPA - Target 70B+. Aligned benchmarks created with different numbers of models. In (a), each point reports the Spearman Rho (ρ) of the best subset of models. In (b), each point shows the Pairwise Ranking Accuracy (Acc_{pair}). Error bars obtained using the sample size from the target set. The dotted line represents the results of alignment using all models.

Target Models	Method	Helpfulness				Honesty			
		Acc_{pair}		ρ		Acc_{pair}		ρ	
		ArmoRM	GPT2	ArmoRM	GPT2	ArmoRM	DPA	ArmoRM	DPA
70B+	No Alignment	0.542 ± 0.010	0.480 ± 0.010	0.154 ± 0.165	0.044 ± 0.251	0.559 ± 0.010	0.567 ± 0.010	0.199 ± 0.164	0.188 ± 0.164
	MetaBench + BenchAlign	0.709 ± 0.009	0.594 ± 0.010	0.570 ± 0.102	0.274 ± 0.146	0.699 ± 0.009	0.668 ± 0.009	0.548 ± 0.106	0.477 ± 0.118
	TinyBenchmarks + BenchAlign	0.725 ± 0.009	0.585 ± 0.010	0.594 ± 0.097	0.244 ± 0.149	0.733 ± 0.009	0.702 ± 0.009	0.622 ± 0.092	0.555 ± 0.104
	BenchAlign	$\mathbf{0.778 \pm 0.008}$	$\mathbf{0.620 \pm 0.010}$	$\mathbf{0.707 \pm 0.074}$	$\mathbf{0.333 \pm 0.139}$	$\mathbf{0.774 \pm 0.008}$	$\mathbf{0.746 \pm 0.009}$	$\mathbf{0.706 \pm 0.074}$	$\mathbf{0.659 \pm 0.084}$
30B+	No Alignment	0.605 ± 0.005	0.477 ± 0.005	0.331 ± 0.114	0.052 ± 0.226	0.618 ± 0.005	0.609 ± 0.005	0.368 ± 0.111	0.319 ± 0.115
	MetaBench + BenchAlign	0.711 ± 0.005	0.564 ± 0.005	0.585 ± 0.075	0.191 ± 0.116	0.725 ± 0.005	0.720 ± 0.005	0.622 ± 0.070	0.608 ± 0.072
	TinyBenchmarks + BenchAlign	0.749 ± 0.005	0.612 ± 0.005	0.666 ± 0.063	0.320 ± 0.106	0.764 ± 0.005	0.756 ± 0.005	0.706 ± 0.057	0.694 ± 0.059
	BenchAlign	$\mathbf{0.765 \pm 0.005}$	$\mathbf{0.637 \pm 0.005}$	$\mathbf{0.710 \pm 0.056}$	$\mathbf{0.387 \pm 0.100}$	$\mathbf{0.773 \pm 0.005}$	$\mathbf{0.763 \pm 0.005}$	$\mathbf{0.730 \pm 0.053}$	$\mathbf{0.713 \pm 0.055}$
13B+	No Alignment	0.583 ± 0.002	0.531 ± 0.002	0.245 ± 0.065	0.094 ± 0.067	0.588 ± 0.002	0.577 ± 0.002	0.251 ± 0.064	0.231 ± 0.065
	MetaBench + BenchAlign	0.744 ± 0.001	0.653 ± 0.002	0.677 ± 0.035	0.442 ± 0.053	0.757 ± 0.001	0.750 ± 0.001	0.706 ± 0.032	0.695 ± 0.033
	TinyBenchmarks + BenchAlign	$\mathbf{0.754 \pm 0.001}$	0.677 ± 0.002	$\mathbf{0.700 \pm 0.033}$	0.505 ± 0.049	$\mathbf{0.770 \pm 0.001}$	$\mathbf{0.770 \pm 0.001}$	$\mathbf{0.735 \pm 0.030}$	$\mathbf{0.741 \pm 0.029}$
	BenchAlign	0.741 ± 0.001	$\mathbf{0.696 \pm 0.002}$	0.674 ± 0.035	$\mathbf{0.552 \pm 0.045}$	0.748 ± 0.001	0.749 ± 0.001	0.686 ± 0.034	0.691 ± 0.034

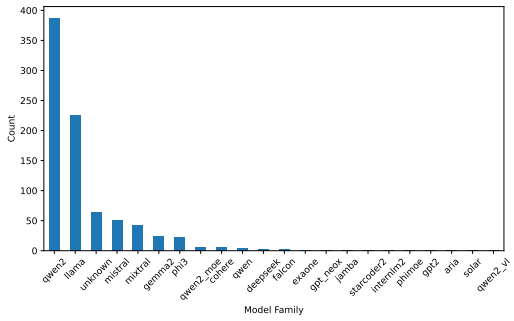
Table 15: Alignment with reduced question pools under model scale splits.

Target	Method	Acc_{pair}				ρ			
		RM_1^{Judge}	RM_2^{Judge}	RM_3^{Judge}	RM_4^{Judge}	RM_1^{Judge}	RM_2^{Judge}	RM_3^{Judge}	RM_4^{Judge}
2%	No Alignment	0.578 ± 0.001	0.743 ± 0.007	0.638 ± 0.015	0.684 ± 0.015	0.272 ± 0.207	0.700 ± 0.122	0.390 ± 0.191	0.511 ± 0.174
	MetaBench + BenchAlign	0.722 ± 0.014	0.808 ± 0.013	0.813 ± 0.012	0.747 ± 0.014	0.601 ± 0.120	0.801 ± 0.065	0.812 ± 0.061	0.648 ± 0.107
	TinyBenchmarks + BenchAlign	0.750 ± 0.014	0.822 ± 0.012	0.819 ± 0.012	0.753 ± 0.014	0.676 ± 0.100	0.830 ± 0.056	0.820 ± 0.059	0.675 ± 0.100
	BenchAlign	$\mathbf{0.768 \pm 0.014}$	$\mathbf{0.842 \pm 0.012}$	$\mathbf{0.820 \pm 0.012}$	$\mathbf{0.773 \pm 0.013}$	$\mathbf{0.698 \pm 0.127}$	$\mathbf{0.855 \pm 0.065}$	$\mathbf{0.824 \pm 0.082}$	$\mathbf{0.693 \pm 0.128}$
5%	No Alignment	0.559 ± 0.001	0.713 ± 0.006	0.610 ± 0.006	0.616 ± 0.006	0.193 ± 0.131	0.606 ± 0.091	0.327 ± 0.124	0.336 ± 0.123
	MetaBench + BenchAlign	0.717 ± 0.006	0.826 ± 0.005	0.779 ± 0.005	0.760 ± 0.005	0.597 ± 0.079	0.836 ± 0.036	0.744 ± 0.054	0.687 ± 0.064
	TinyBenchmarks + BenchAlign	0.760 ± 0.005	0.841 ± 0.005	0.808 ± 0.005	$\mathbf{0.803 \pm 0.005}$	0.694 ± 0.063	0.859 ± 0.031	0.801 ± 0.043	$\mathbf{0.781 \pm 0.047}$
	BenchAlign	$\mathbf{0.770 \pm 0.005}$	$\mathbf{0.850 \pm 0.005}$	$\mathbf{0.811 \pm 0.005}$	$\mathbf{0.803 \pm 0.005}$	$\mathbf{0.705 \pm 0.074}$	$\mathbf{0.873 \pm 0.036}$	$\mathbf{0.806 \pm 0.052}$	0.770 ± 0.060
10%	No Alignment	0.563 ± 0.003	0.713 ± 0.003	0.605 ± 0.003	0.626 ± 0.003	0.184 ± 0.094	0.601 ± 0.000	0.314 ± 0.087	0.363 ± 0.084
	MetaBench + BenchAlign	0.752 ± 0.003	0.826 ± 0.002	0.793 ± 0.003	0.772 ± 0.003	0.687 ± 0.047	0.840 ± 0.026	0.774 ± 0.035	0.724 ± 0.042
	TinyBenchmarks + BenchAlign	0.772 ± 0.003	0.846 ± 0.002	0.810 ± 0.002	$\mathbf{0.808 \pm 0.003}$	0.731 ± 0.041	0.872 ± 0.021	0.810 ± 0.030	$\mathbf{0.800 \pm 0.031}$
	BenchAlign	$\mathbf{0.782 \pm 0.003}$	$\mathbf{0.850 \pm 0.002}$	$\mathbf{0.814 \pm 0.002}$	$\mathbf{0.808 \pm 0.003}$	$\mathbf{0.746 \pm 0.045}$	$\mathbf{0.875 \pm 0.024}$	$\mathbf{0.814 \pm 0.034}$	0.796 ± 0.037

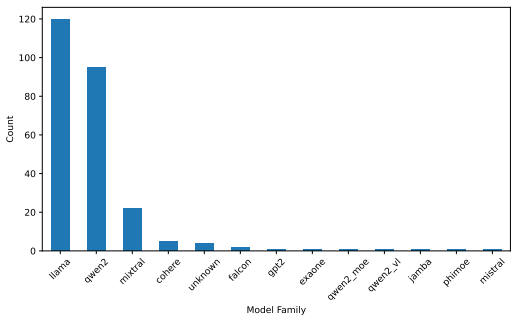
Table 16: Alignment with reduced question pools under heterogeneous preferences

Target Models	Method	Helpfulness				Honesty			
		Acc_{pair}		ρ		Acc_{pair}		ρ	
		ArmoRM	GPT2	ArmoRM	GPT2	ArmoRM	DPA	ArmoRM	DPA
2%	No Alignment	0.720 ± 0.014	0.633 ± 0.015	0.614 ± 0.151	0.383 ± 0.196	0.736 ± 0.014	0.724 ± 0.014	0.652 ± 0.140	0.627 ± 0.147
	MetaBench + BenchAlign	0.831 ± 0.012	0.779 ± 0.013	0.842 ± 0.052	0.741 ± 0.082	0.852 ± 0.011	0.842 ± 0.012	0.872 ± 0.043	0.863 ± 0.045
	TinyBenchmarks + BenchAlign	$\mathbf{0.869 \pm 0.011}$	$\mathbf{0.811 \pm 0.013}$	$\mathbf{0.895 \pm 0.035}$	0.796 ± 0.066	0.881 ± 0.010	0.874 ± 0.011	0.915 ± 0.029	0.910 ± 0.030
	BenchAlign	0.866 ± 0.011	0.807 ± 0.013	$\mathbf{0.895 \pm 0.035}$	$\mathbf{0.800 \pm 0.078}$	$\mathbf{0.891 \pm 0.010}$	$\mathbf{0.891 \pm 0.010}$	$\mathbf{0.933 \pm 0.028}$	$\mathbf{0.929 \pm 0.030}$
5%	No Alignment	0.708 ± 0.006	0.605 ± 0.006	0.580 ± 0.095	0.314 ± 0.125	0.727 ± 0.006	0.722 ± 0.006	0.630 ± 0.088	0.616 ± 0.090
	MetaBench + BenchAlign	0.808 ± 0.005	0.746 ± 0.006	0.802 ± 0.043	0.673 ± 0.067	0.827 ± 0.005	0.823 ± 0.005	0.843 ± 0.035	0.834 ± 0.037
	TinyBenchmarks + BenchAlign	$\mathbf{0.851 \pm 0.005}$	0.791 ± 0.005	$\mathbf{0.867 \pm 0.030}$	$\mathbf{0.764 \pm 0.050}$	0.864 ± 0.004	0.863 ± 0.004	0.899 ± 0.023	0.897 ± 0.023
	BenchAlign	0.849 ± 0.005	$\mathbf{0.792 \pm 0.005}$	0.866 ± 0.034	0.762 ± 0.056	$\mathbf{0.870 \pm 0.004}$	$\mathbf{0.867 \pm 0.004}$	$\mathbf{0.905 \pm 0.024}$	$\mathbf{0.899 \pm 0.026}$
10%	No Alignment	0.678 ± 0.003	0.590 ± 0.003	0.502 ± 0.074	0.264 ± 0.090	0.697 ± 0.003	0.695 ± 0.003	0.558 ± 0.068	0.551 ± 0.069
	MetaBench + BenchAlign	0.820 ± 0.002	0.769 ± 0.003	0.825 ± 0.028	0.713 ± 0.043	0.838 ± 0.002	0.836 ± 0.002	0.859 ± 0.023	0.857 ± 0.023
	TinyBenchmarks + BenchAlign	$\mathbf{0.855 \pm 0.002}$	0.808 ± 0.003	$\mathbf{0.876 \pm 0.020}$	0.793 ± 0.032	$\mathbf{0.868 \pm 0.002}$	$\mathbf{0.866 \pm 0.002}$	$\mathbf{0.903 \pm 0.016}$	$\mathbf{0.902 \pm 0.016}$
	BenchAlign	$\mathbf{0.855 \pm 0.002}$	$\mathbf{0.814 \pm 0.002}$	$\mathbf{0.876 \pm 0.020}$	$\mathbf{0.804 \pm 0.033}$	0.863 ± 0.002	0.862 ± 0.002	0.894 ± 0.019	0.891 ± 0.019

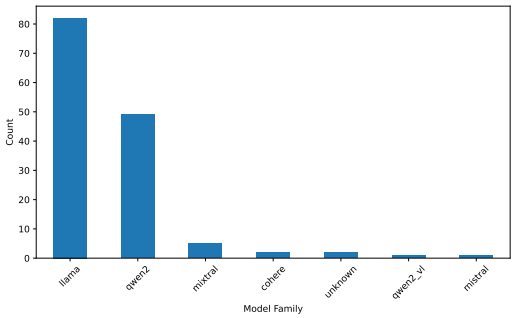
Table 17: Alignment with reduced question pools under arbitrary model sets



(a) Target 13B+



(b) Target 30B+



(c) Target 70B+

Figure 38: Distribution of models by model family across model size targets

I Finding the Best Subset of Models

I.1 Model Family

We study whether the model family[20] drives alignment by performing an experiment inspired by the most common architectures found on each target by model scale. We select the top 5 families from each target and created a new group of train/test splits that allow us to test whether the model family is relevant for alignment:

- Llama
- Qwen2

The test samples consisted of all Llama or Qwen models found in the target set as they are the most common across all target sets (See Figure 38). Then, we find the top 5 biggest families on train:

- Qwen2
- Llama
- Mistral
- Gemma
- Phi

We randomly sample models from each of these 5 groups making sure they all share the same size to ensure all had the same number of models[21]. Finally, we evaluate for all reward models from Section 4.2 whether the best family for alignment is the same family as the target.

Table 18 shows the results of our experiments. Surprisingly, the model family with the best results coincides with the target in 45.83% of the experiments. In the Llama experiments, this coincidence

[20]We consider a model family as the one obtained from the HuggingFace metadata entry with the same name.

[21]We fixed this number as 64 as that is the number of Phi models, the smallest group from the top 5

Table 18: Best model families in family target experiments

RM	Target	Train Family	Test Family
ArmoRM (Helpful)	13B+	qwen	llama
ArmoRM (Helpful)	30B+	qwen	llama
ArmoRM (Helpful)	70B+	llama	llama
ArmoRM (Helpful)	13B+	mistral	qwen
ArmoRM (Helpful)	30B+	qwen	qwen
ArmoRM (Helpful)	70B+	qwen	qwen
GPT2	13B+	llama	llama
GPT2	30B+	qwen	llama
GPT2	70B+	gemma	llama
GPT2	13B+	llama	qwen
GPT2	30B+	qwen	qwen
GPT2	70B+	qwen	qwen
ArmoRM (Honest)	13B+	llama	llama
ArmoRM (Honest)	30B+	qwen	llama
ArmoRM (Honest)	70B+	mistral	llama
ArmoRM (Honest)	13B+	mistral	qwen
ArmoRM (Honest)	30B+	qwen	qwen
ArmoRM (Honest)	70B+	mistral	qwen
DPA	13B+	llama	llama
DPA	30B+	llama	llama
DPA	70B+	mistral	llama
DPA	13B+	mistral	qwen
DPA	30B+	qwen	qwen
DPA	70B+	mistral	qwen

Target Models	Method	Helpfulness				Honesty			
		Acc_{pair}		ρ		Acc_{pair}		ρ	
		ArmoRM	GPT2	ArmoRM	GPT2	ArmoRM	DPA	ArmoRM	DPA
Llama + Qwen	NO ALIGNMENT	0.709 ± 0.002	0.606 ± 0.002	0.597 ± 0.045	0.312 ± 0.064	0.732 ± 0.002	0.738 ± 0.002	0.654 ± 0.040	0.667 ± 0.038
	RANDOM	0.676 ± 0.002	0.605 ± 0.002	0.393 ± 0.060	0.310 ± 0.064	0.708 ± 0.002	0.711 ± 0.002	0.604 ± 0.044	0.611 ± 0.044
	INDIVIDUAL	0.733 ± 0.002	0.644 ± 0.002	0.648 ± 0.040	0.397 ± 0.059	0.757 ± 0.002	0.761 ± 0.002	0.705 ± 0.035	0.712 ± 0.034
	METABENCH	0.708 ± 0.001	0.602 ± 0.001	0.594 ± 0.047	0.302 ± 0.066	0.731 ± 0.001	0.736 ± 0.001	0.651 ± 0.042	0.664 ± 0.041
	TINYBENCHMARKS	0.697 ± 0.001	0.600 ± 0.001	0.573 ± 0.048	0.296 ± 0.066	0.720 ± 0.001	0.727 ± 0.001	0.633 ± 0.043	0.646 ± 0.042
	BENCHALIGN	$\mathbf{0.803 \pm 0.002}$	$\mathbf{0.702 \pm 0.002}$	$\mathbf{0.805 \pm 0.024}$	$\mathbf{0.583 \pm 0.046}$	$\mathbf{0.819 \pm 0.001}$	$\mathbf{0.828 \pm 0.001}$	$\mathbf{0.838 \pm 0.020}$	$\mathbf{0.850 \pm 0.019}$

Table 19: Alignment with Llama and Qwen older generation models vs newer ones

happens 41.6% of the time, while 50% of the times in the Qwen experiments. This results suggest that the choice of model family for experiments is not very different from choosing a family at random.

I.2 Model Generation

We use BenchAlign to rerank IFEval using older Llama and Qwen generations to predict future ones. Our results in Tables 19, 20 and 21 show that using models from older generations can create somewhat generalizable benchmarks for future generations. However, our results don't cover the vast majority of model families currently available nor the state-of-the-art Qwen models due to the closure of OpenLLMLeaderboard. Therefore, our findings should be used to guide future research on finding better subset rules rather than direct practical applications.

- Source: Llama 2 and 3 models | Target: Llama 3.x models
- Source: Qwen 2 models | Target: Qwen 2.x models
- Source: Llama 2, 3 and Qwen 2 models | Target: Llama 3.x and Qwen 2.x models

Target Models	Method	Helpfulness				Honesty			
		Acc_{pair}		ρ		Acc_{pair}		ρ	
		ArmoRM	GPT2	ArmoRM	GPT2	ArmoRM	DPA	ArmoRM	DPA
Qwen	NO ALIGNMENT	0.730 ± 0.003	0.614 ± 0.004	0.651 ± 0.054	0.323 ± 0.087	0.756 ± 0.003	0.757 ± 0.003	0.716 ± 0.046	0.721 ± 0.045
	RANDOM	0.714 ± 0.003	0.618 ± 0.004	0.394 ± 0.082	0.333 ± 0.087	0.729 ± 0.003	0.732 ± 0.003	0.661 ± 0.053	0.670 ± 0.052
	INDIVIDUAL	0.767 ± 0.003	0.642 ± 0.004	0.702 ± 0.048	0.393 ± 0.082	0.779 ± 0.003	0.791 ± 0.003	0.753 ± 0.041	0.762 ± 0.039
	METABENCH	0.736 ± 0.002	0.616 ± 0.003	0.655 ± 0.058	0.320 ± 0.091	0.759 ± 0.002	0.762 ± 0.002	0.715 ± 0.050	0.720 ± 0.049
	TINYBENCHMARKS	0.712 ± 0.002	0.611 ± 0.003	0.624 ± 0.062	0.313 ± 0.091	0.738 ± 0.002	0.739 ± 0.002	0.692 ± 0.053	0.698 ± 0.052
	BENCHALIGN	$\mathbf{0.839 \pm 0.003}$	$\mathbf{0.684 \pm 0.003}$	$\mathbf{0.858 \pm 0.024}$	$\mathbf{0.546 \pm 0.067}$	$\mathbf{0.856 \pm 0.003}$	$\mathbf{0.864 \pm 0.003}$	$\mathbf{0.892 \pm 0.019}$	$\mathbf{0.904 \pm 0.017}$

Table 20: Alignment with Qwen older generation models vs newer ones

Target Models	Method	Helpfulness				Honesty			
		Acc_{pair}		ρ		Acc_{pair}		ρ	
		ArmoRM	GPT2	ArmoRM	GPT2	ArmoRM	DPA	ArmoRM	DPA
Llama	NO ALIGNMENT	0.727 ± 0.004	0.676 ± 0.004	0.623 ± 0.060	0.510 ± 0.073	0.736 ± 0.003	0.740 ± 0.003	0.648 ± 0.057	0.649 ± 0.057
	RANDOM	0.714 ± 0.004	0.661 ± 0.004	0.516 ± 0.073	0.445 ± 0.080	0.699 ± 0.004	0.703 ± 0.004	0.563 ± 0.067	0.569 ± 0.067
	INDIVIDUAL	0.741 ± 0.003	0.691 ± 0.004	0.643 ± 0.058	0.535 ± 0.071	0.750 ± 0.003	0.756 ± 0.003	0.665 ± 0.054	0.671 ± 0.054
	METABENCH	0.726 ± 0.002	0.673 ± 0.003	0.624 ± 0.064	0.506 ± 0.078	0.735 ± 0.002	0.737 ± 0.002	0.650 ± 0.061	0.649 ± 0.061
	TINYBENCHMARKS	0.710 ± 0.002	0.669 ± 0.003	0.588 ± 0.067	0.489 ± 0.078	0.717 ± 0.002	0.723 ± 0.002	0.612 ± 0.064	0.615 ± 0.064
	BENCHALIGN	$\mathbf{0.795 \pm 0.003}$	$\mathbf{0.748 \pm 0.003}$	$\mathbf{0.777 \pm 0.038}$	$\mathbf{0.694 \pm 0.051}$	$\mathbf{0.805 \pm 0.003}$	$\mathbf{0.811 \pm 0.003}$	$\mathbf{0.801 \pm 0.035}$	$\mathbf{0.814 \pm 0.033}$

Table 21: Alignment with Llama older generation models vs newer ones

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Yes, the claims made in the abstract and introduction are accurately reflected throughout the paper.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Yes, please see Section 5 for the limitations.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

Justification: We do not include theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: We explain the datasets, model architecture and hyperparameters for our experiments in Section 4. Code will be released following the review process.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We use datasets and reward models publicly available. Information to reproduce our experiments can be found in Section 4.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: We specify all data splits, hyperparameters and optimizer in Section 4.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: All of our experiments report error bars where applicable.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Please see appendix D.5.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: We followed the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Please see Appendix A.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: We do not introduce data or models that pose a high risk of misuse.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The creators or original asset owners have been credited in appropriate ways.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: In this work, no new assets are released.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: This work does not involve crowdsourcing nor research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: This work does not involve crowdsourcing nor research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: The core methodology, scientific rigorousness, or originality of the research does not involve the usage of LLMs.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.